In [6]:
```
import numpy as np

a = np.eye(4)
print(a)
print("\n")

b = np.arange(5, 20, 3)
print(b)
print("\n")

c = np.linspace(10, 20, 6)
print(c)
print("\n")
+
d = np.random.rand(4)
print(d)
print("\n")

e = np.random.randint(10, 50, 6)
print(e)
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]


[ 5  8 11 14 17]


[10. 12. 14. 16. 18. 20.]


[0.46329099 0.49236886 0.72397955 0.38182704]


[45 42 19 43 28 38]
```

In [35]:
```
# array inspection functions

import numpy as np

f = np.shape([[2, 4, 6], [8, 10, 12]])
print(f)

g = np.array([[9, 3, 7], [1, 5, 11]])
h = np.shape(g)
print(h)

i = np.size(g)
print(i)

j = np.ndim(g)
print(j)

print(type(g))
```

```
(2, 3)
(2, 3)
6
2
<class 'numpy.ndarray'>
```

In [3]:
```
#array manipulation
k=np.transpose(g)
print(k)
```

```
[[ 9  1]
 [ 3  5]
 [ 7 11]]
```

In [4]:
```
import numpy as np

k = np.array([[12, 4],
              [6, 10]])
l = np.array([[3, 2],
              [1, 5]])
m = np.add(k, l)
print(m)
n = np.subtract(k, l)
print(n)
o = np.multiply(k, l)
print(o)
p = np.divide(k, l)
print(p)
q = np.sqrt(l)
print(q)
r = np.log(l)
print(r)
```

```
[[15  6]
 [ 7 15]]
[[9 2]
 [5 5]]
[[36  8]
 [ 6 50]]
[[4. 2.]
 [6. 2.]]
[[1.73205081 1.41421356]
 [1.         2.23606798]]
[[1.09861229 0.69314718]
 [0.         1.60943791]]
```

In [7]:
```
import numpy as np

# statistical functions
l = np.array([[12, 6],
              [3, 15]])

s = np.min(l)
print(s, "\n")

t = np.max(l)
print(t, "\n")

u = np.mean(l)
v = np.median(l)
w = np.std(l)
```

```
x = np.var(l)

print(u, "\n")
print(v, "\n")
print(w, "\n")
print(x, "\n")

y = np.array([10, 12, 14, 16, 18])
z = np.mean(y)
print(z)
```

```
3

15

9.0

9.0

4.743416490252569

22.5

14.0
```

In [8]:
```
#sorting and searching funcs
a=np.array([95,48,90,5,4,3,66,2])
np.sort(a)
```

Out[8]:
```
array([ 2,  3,  4,  5, 48, 66, 90, 95])
```

In [9]:
```
ab=np.array([8,8,9,4,6,6,1,1])
np.unique(ab)
```

Out[9]:
```
array([1, 4, 6, 8, 9])
```

In [13]:
```
import pandas as pd
bb=pd.read_csv("iris.csv")
bb.head()
```

Out[13]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

In [14]:
```
bb.tail()
```

Out[14]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

In [15]:
```
bb.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    int64  
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

In [16]:
```
bb.describe()
```

Out[16]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

In [17]:
```
bb.shape
```

Out[17]:
```
(150, 5)
```

In [19]:
```
bb.columns
```

Out[19]:
```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

In [22]:
```
df[['sepal length (cm)', 'sepal width (cm)']]
```

Out[22]:

	sepal length (cm)	sepal width (cm)
0	5.1	3.5
1	4.9	3.0
2	4.7	3.2
3	4.6	3.1
4	5.0	3.6
...	...	...
145	6.7	3.0
146	6.3	2.5
147	6.5	3.0
148	6.2	3.4
149	5.9	3.0

150 rows × 2 columns

In [24]:
```
bb.loc[1,'sepal length (cm)']
```

Out[24]:
```
np.float64(4.9)
```

In [25]:
```
 #bb.drop()
#bb.rename()
#bb.replace()
```

In [26]:
```
bb.to_csv("iris.csv")
```

In [31]:
```
bb = df
bb.to_excel("bro.xlsx", index=False)
```

In [32]:
```
bb.groupby('sepal length (cm)')
```

Out[32]:
```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x00000281CB24BF80>
```

In [33]:
```
import pandas as pd

c = pd.DataFrame({
    'Data': [10, 12, 14, 16, 18],
    'age': [22, 23, 25, 26, 28]
})

pd.concat([bb, c])
```

Out[33]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species	Data	age
0	5.1	3.5	1.4	0.2	0.0	NaN	NaN
1	4.9	3.0	1.4	0.2	0.0	NaN	NaN
2	4.7	3.2	1.3	0.2	0.0	NaN	NaN
3	4.6	3.1	1.5	0.2	0.0	NaN	NaN
4	5.0	3.6	1.4	0.2	0.0	NaN	NaN
...	...	...	...	...	...	...	...
0	NaN	NaN	NaN	NaN	NaN	10.0	22.0
1	NaN	NaN	NaN	NaN	NaN	12.0	23.0
2	NaN	NaN	NaN	NaN	NaN	14.0	25.0
3	NaN	NaN	NaN	NaN	NaN	16.0	26.0
4	NaN	NaN	NaN	NaN	NaN	18.0	28.0

155 rows × 7 columns

In [36]:
```
bb.join(c)
```

Out[36]:

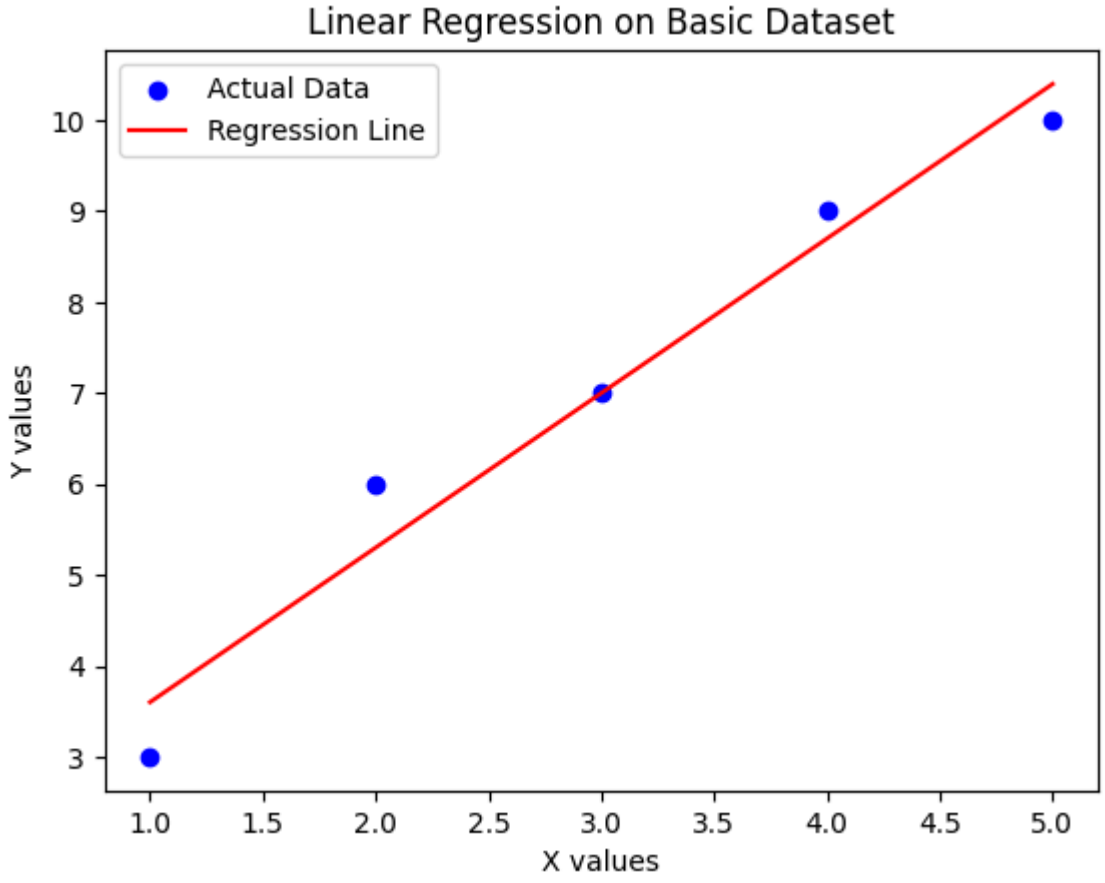
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species	Data	age
0	5.1	3.5	1.4	0.2	0	10.0	22.0
1	4.9	3.0	1.4	0.2	0	12.0	23.0
2	4.7	3.2	1.3	0.2	0	14.0	25.0
3	4.6	3.1	1.5	0.2	0	16.0	26.0
4	5.0	3.6	1.4	0.2	0	18.0	28.0
...	...	...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2	NaN	NaN
146	6.3	2.5	5.0	1.9	2	NaN	NaN
147	6.5	3.0	5.2	2.0	2	NaN	NaN
148	6.2	3.4	5.4	2.3	2	NaN	NaN
149	5.9	3.0	5.1	1.8	2	NaN	NaN

150 rows × 7 columns

In [1]:
```
#Linear Regression
```

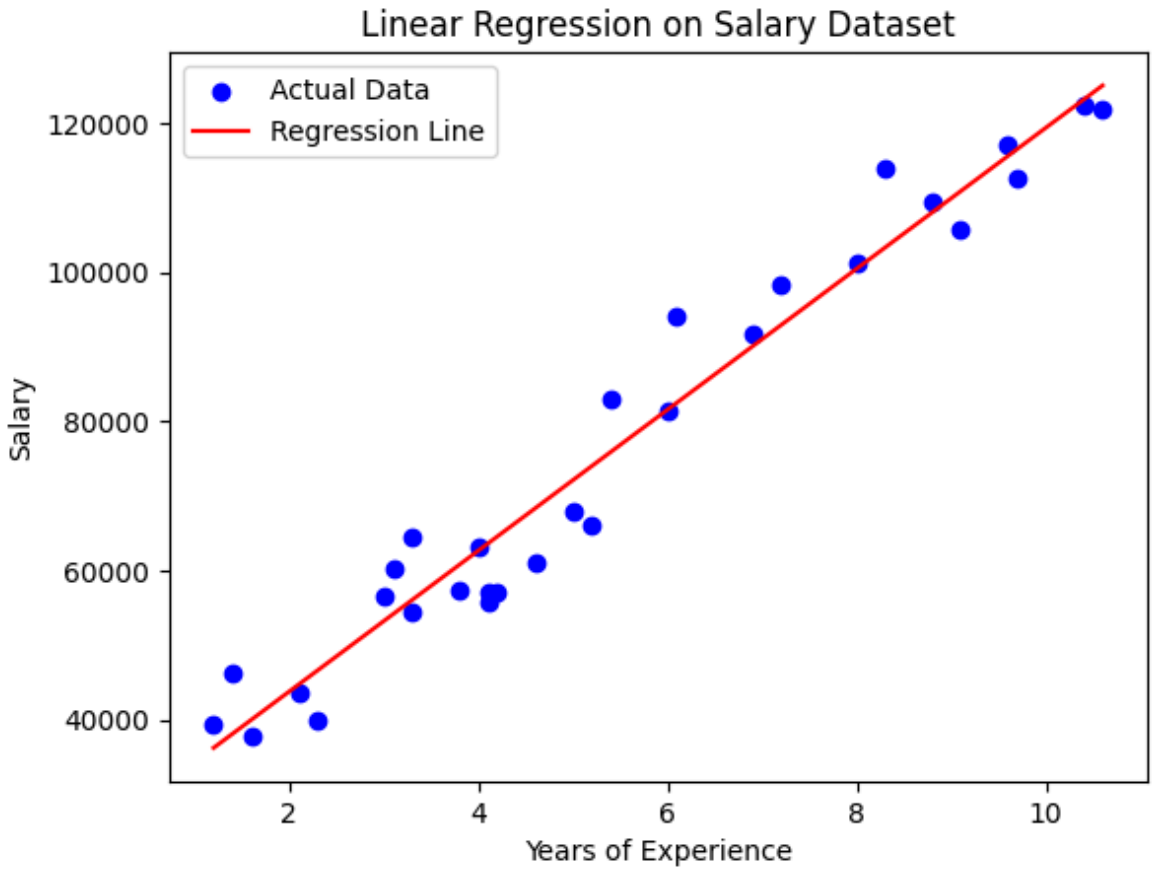
In [14]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Your original basic dataset
X = np.array([1, 2, 3, 4, 5])
Y = np.array([3, 6, 7, 9, 10])

x_mean = np.mean(X)
y_mean = np.mean(Y)

num = np.sum((X - x_mean) * (Y - y_mean))
den = np.sum((X - x_mean) ** 2)
beta1 = num / den
beta0 = y_mean - beta1 * x_mean

print("Slope (beta1): ", beta1)
print("Intercept (beta0): ", beta0)

Y_pred = beta0 + beta1 * X

plt.scatter(X, Y, color="blue", label="Actual Data")
plt.plot(X, Y_pred, color="red", label="Regression Line")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Linear Regression on Basic Dataset")
plt.legend()
plt.show()
```

```
Slope (beta1):  1.7
Intercept (beta0):  1.9000000000000004
```

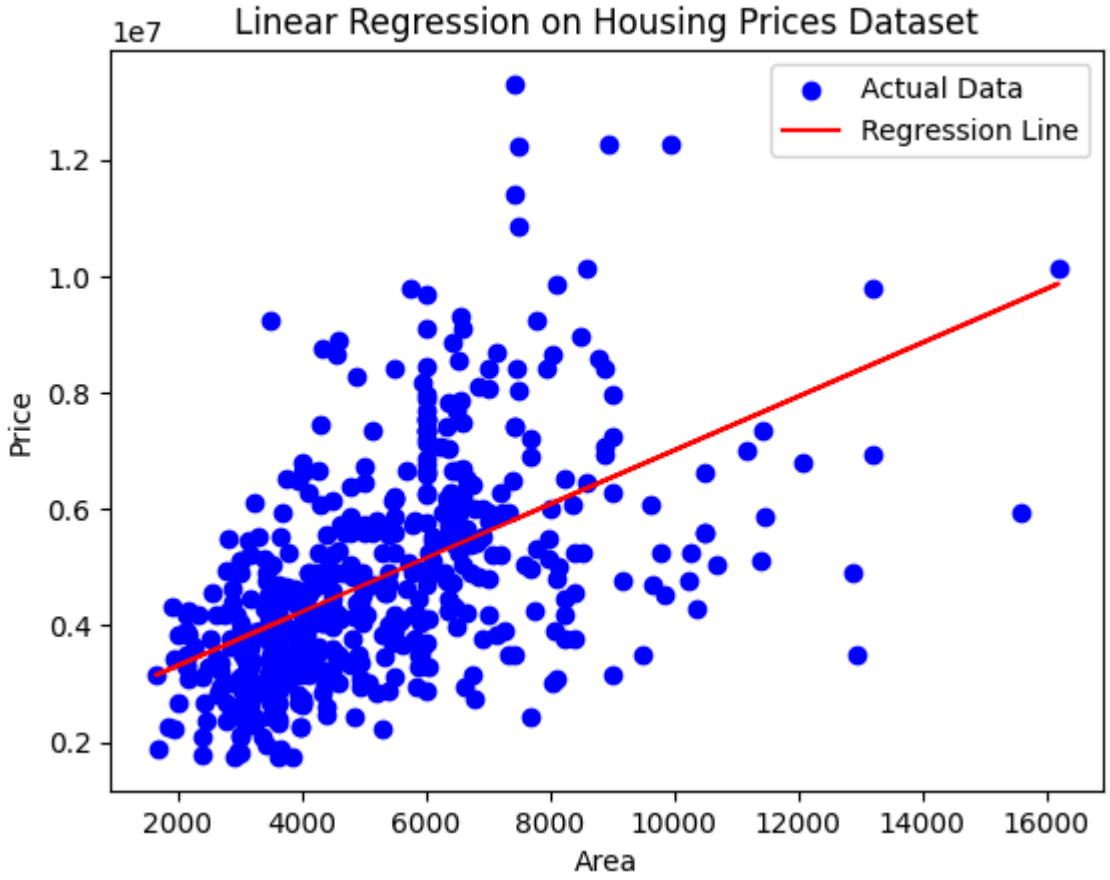


In [15]:
```
# Your original Salary dataset (Kaggle: https://www.kaggle.com/datasets/abhishek
# Save as Salary.csv
data = pd.read_csv('Salary_dataset(program1).csv')
X_salary = data["YearsExperience"].values
Y_salary = data["Salary"].values

x_mean_salary = np.mean(X_salary)
y_mean_salary = np.mean(Y_salary)

num_salary = np.sum((X_salary - x_mean_salary) * (Y_salary - y_mean_salary))
den_salary = np.sum((X_salary - x_mean_salary) ** 2)
beta1_salary = num_salary / den_salary
beta0_salary = y_mean_salary - beta1_salary * x_mean_salary

print("Slope (beta1):", beta1_salary)
print("Intercept (beta0):", beta0_salary)

Y_pred_salary = beta0_salary + beta1_salary * X_salary

plt.scatter(X_salary, Y_salary, color="blue", label="Actual Data")
plt.plot(X_salary, Y_pred_salary, color="red", label="Regression Line")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Linear Regression on Salary Dataset")
plt.legend()
plt.show()
```

```
Slope (beta1): 9449.962321455076
Intercept (beta0): 24848.2039665232
```



In [16]:
```
# Second dataset for Linear Regression (Kaggle: https://www.kaggle.com/datasets/
# Download and save as housing.csv
housing = pd.read_csv('Housing_dataset(program1).csv')
X_housing = housing['area'].values          # feature
Y_housing = housing['price'].values         # target

# Same manual linear regression code as you used
x_mean_h = np.mean(X_housing)
y_mean_h = np.mean(Y_housing)
num_h = np.sum((X_housing - x_mean_h) * (Y_housing - y_mean_h))
den_h = np.sum((X_housing - x_mean_h) ** 2)
beta1_h = num_h / den_h
beta0_h = y_mean_h - beta1_h * x_mean_h

Y_pred_h = beta0_h + beta1_h * X_housing

plt.scatter(X_housing, Y_housing, color="blue", label="Actual Data")
plt.plot(X_housing, Y_pred_h, color="red", label="Regression Line")
plt.xlabel("Area")
plt.ylabel("Price")
plt.title("Linear Regression on Housing Prices Dataset")
plt.legend()
plt.show()
```



In [17]:
```
#Logistic Regression
```

In [18]:
```
X_log = np.array([1, 2, 3, 4, 5])
y_log = np.array([0, 0, 0, 1, 1])

w = 0.0
b = 0.0
lr = 0.1
epochs = 1000

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

for _ in range(epochs):
    z = w * X_log + b
    y_pred = sigmoid(z)
    dw = np.mean((y_pred - y_log) * X_log)
    db = np.mean(y_pred - y_log)
    w = w - lr * dw
    b = b - lr * db

print("Weight:", w)
print("Bias:", b)

def predict_logistic(X):
    probs = sigmoid(w * X + b)
    return np.where(probs >= 0.5, 1, 0)

print("Predicted Output:", predict_logistic(X_log))
```

```
Weight: 1.769045864973106
Bias: -5.956350539118638
Predicted Output: [0 0 0 1 1]
```

In [19]:
```python
# Logistic Regression - Heart Disease (Kaggle: https://www.kaggle.com/datasets/j
heart = pd.read_csv('heart_dataset(program1).csv')
X_heart = heart.drop('target', axis=1).values
y_heart = heart['target'].values

# Simple Logistic Regression with sklearn (standard way)
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X_heart, y_heart, test_size=
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train, y_train)
y_pred_heart = logreg.predict(X_test)
print("Heart Logistic Accuracy:", accuracy_score(y_test, y_pred_heart))
```

```
Heart Logistic Accuracy: 0.8051948051948052
```

In [20]:
```python
# Logistic Regression - Pima Diabetes (Kaggle: https://www.kaggle.com/datasets/u
diabetes = pd.read_csv('diabetes_ds(program1).csv')
X_dia = diabetes.drop('Outcome', axis=1).values
y_dia = diabetes['Outcome'].values

X_train, X_test, y_train, y_test = train_test_split(X_dia, y_dia, test_size=0.3,
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train, y_train)
y_pred_dia = logreg.predict(X_test)
print("Diabetes Logistic Accuracy:", accuracy_score(y_test, y_pred_dia))
```

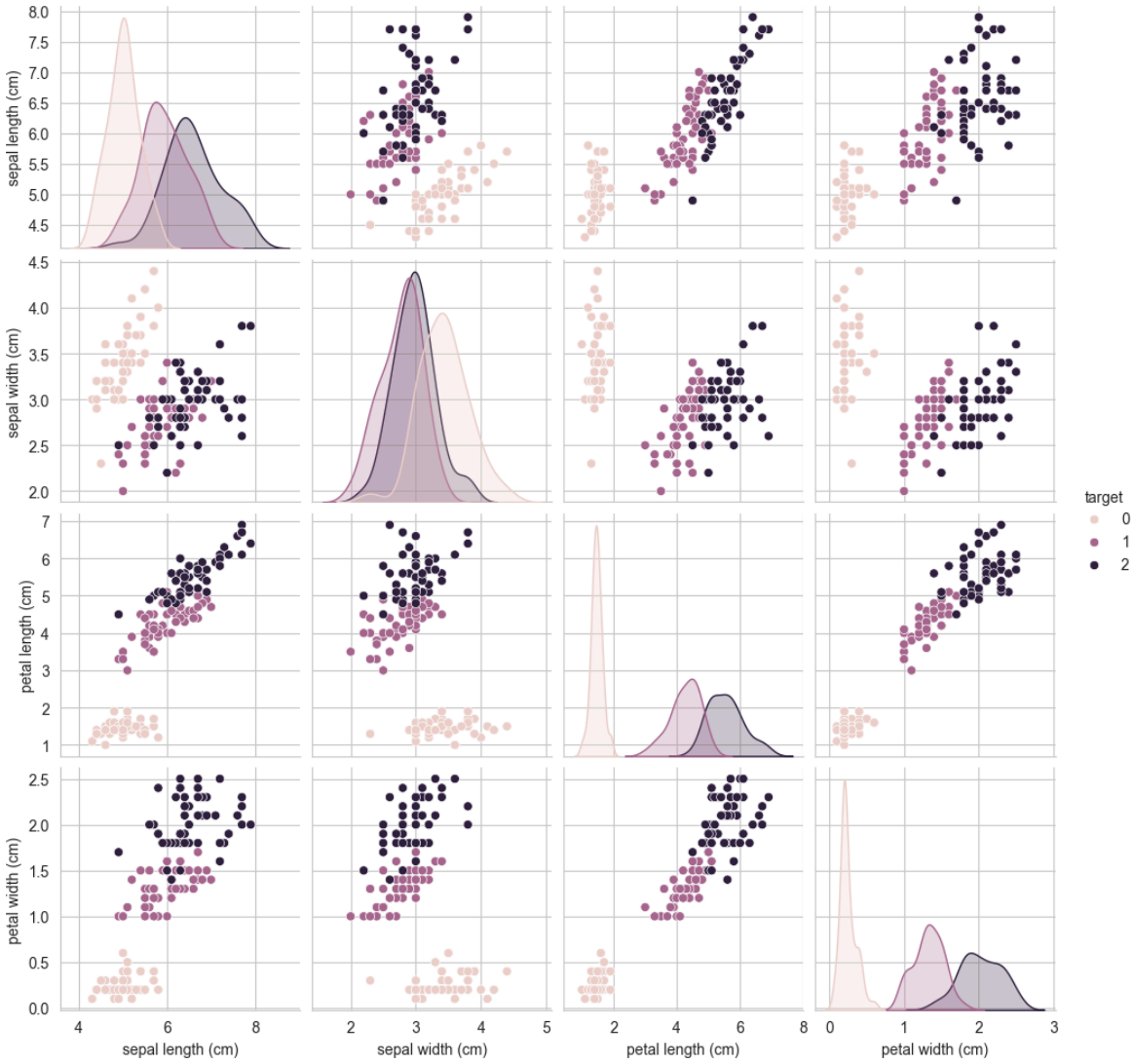
```
Diabetes Logistic Accuracy: 0.7359307359307359
```

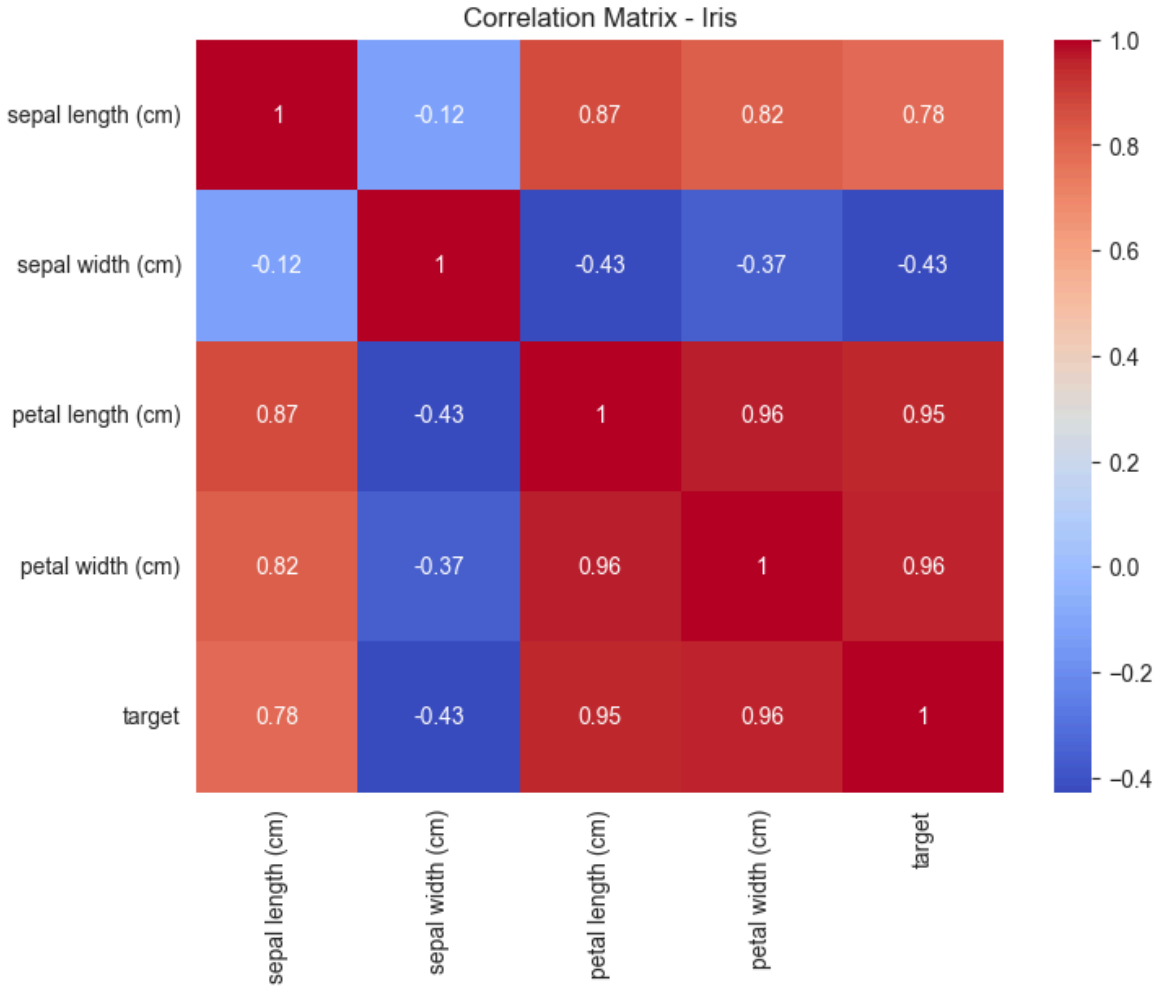
In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_rep

import warnings
warnings.filterwarnings('ignore')
sns.set_style("whitegrid")
```

In [2]:
```
# Iris dataset
from sklearn.datasets import load_iris
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

print(df.head())
print("\nDataset shape:", df.shape)
print("\nDataset Info:")
print(df.info())
print("\nStatistical Summary:")
print(df.describe())

# Check missing values
print("\nMissing values:")
print(df.isnull().sum())

sns.pairplot(df, hue="target")
plt.show()

# Correlation heatmap
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix - Iris")
plt.show()

# Prepare data
X = df.drop('target', axis=1)
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_

# Scale (for KNN & SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 1. Decision Tree
dt = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
print("\nDecision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))

# 2. KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)
print("\nKNN Accuracy:", accuracy_score(y_test, y_pred_knn))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))

# 3. Naive Bayes
nb = GaussianNB()
nb.fit(X_train, y_train)
y_pred_nb = nb.predict(X_test)
print("\nNaive Bayes Accuracy:", accuracy_score(y_test, y_pred_nb))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_nb))
print(classification_report(y_test, y_pred_nb))

# 4. SVM
svm = SVC(kernel='linear', random_state=42)
svm.fit(X_train_scaled, y_train)
y_pred_svm = svm.predict(X_test_scaled)
print("\nSVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0

Dataset shape: (150, 5)

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
None

Statistical Summary:
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000
mean            5.843333          3.057333           3.758000
std             0.828066          0.435866           1.765298
min             4.300000          2.000000           1.000000
25%             5.100000          2.800000           1.600000
50%             5.800000          3.000000           4.350000
75%             6.400000          3.300000           5.100000
max             7.900000          4.400000           6.900000

       petal width (cm)      target
count        150.000000  150.000000
mean           1.199333    1.000000
std            0.762238    0.819232
min            0.100000    0.000000
25%            0.300000    0.000000
50%            1.300000    1.000000
75%            1.800000    2.000000
max            2.500000    2.000000

Missing values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64
```


sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)
target
0
1
2


Correlation Matrix - Iris
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)
target
1
-0.12
0.87
0.82
0.78
-0.12
1
-0.43
-0.37
-0.43
0.87
-0.43
1
0.96
0.95
0.82
-0.37
0.96
1
0.96
0.78
-0.43
0.95
0.96
1
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)
target
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

```
Decision Tree Accuracy: 0.9777777777777777
Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  1 12]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       0.93      1.00      0.96        13
           2       1.00      0.92      0.96        13

    accuracy                           0.98        45
   macro avg       0.98      0.97      0.97        45
weighted avg       0.98      0.98      0.98        45


KNN Accuracy: 1.0
Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45


Naive Bayes Accuracy: 0.9777777777777777
Confusion Matrix:
 [[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      0.92      0.96        13
           2       0.93      1.00      0.96        13

    accuracy                           0.98        45
   macro avg       0.98      0.97      0.97        45
weighted avg       0.98      0.98      0.98        45


SVM Accuracy: 0.9777777777777777
Confusion Matrix:
 [[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      0.92      0.96        13
           2       0.93      1.00      0.96        13

    accuracy                           0.98        45
```

```
   macro avg       0.98      0.97      0.97        45
weighted avg       0.98      0.98      0.98        45
```

In [6]:

```
# Heart Disease dataset
# Load dataset (Kaggle: johnsmith88/heart-disease-dataset)
heart = pd.read_csv('heart_ds.csv')

print(heart.head())
print("\nDataset shape:", heart.shape)
print("\nDataset Info:")
print(heart.info())
print("\nStatistical Summary:")
print(heart.describe())

# Missing values
print("\nMissing values:")
print(heart.isnull().sum())

# Pairplot
sns.pairplot(heart, hue="target")
plt.show()

# Correlation heatmap
plt.figure(figsize=(10,8))
sns.heatmap(heart.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Matrix - Heart Disease")
plt.show()

# Prepare data
X = heart.drop('target', axis=1)
y = heart['target']

# Label encoding (in case any object columns - matches your program4 style)
for col in X.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_

# Scale
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Decision Tree
dt = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
print("\nDecision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)
print("\nKNN Accuracy:", accuracy_score(y_test, y_pred_knn))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
```

```
print(classification_report(y_test, y_pred_knn))

# Naive Bayes
nb = GaussianNB()
nb.fit(X_train, y_train)
y_pred_nb = nb.predict(X_test)
print("\nNaive Bayes Accuracy:", accuracy_score(y_test, y_pred_nb))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_nb))
print(classification_report(y_test, y_pred_nb))

# SVM
svm = SVC(kernel='linear', random_state=42)
svm.fit(X_train_scaled, y_train)
y_pred_svm = svm.predict(X_test_scaled)
print("\nSVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

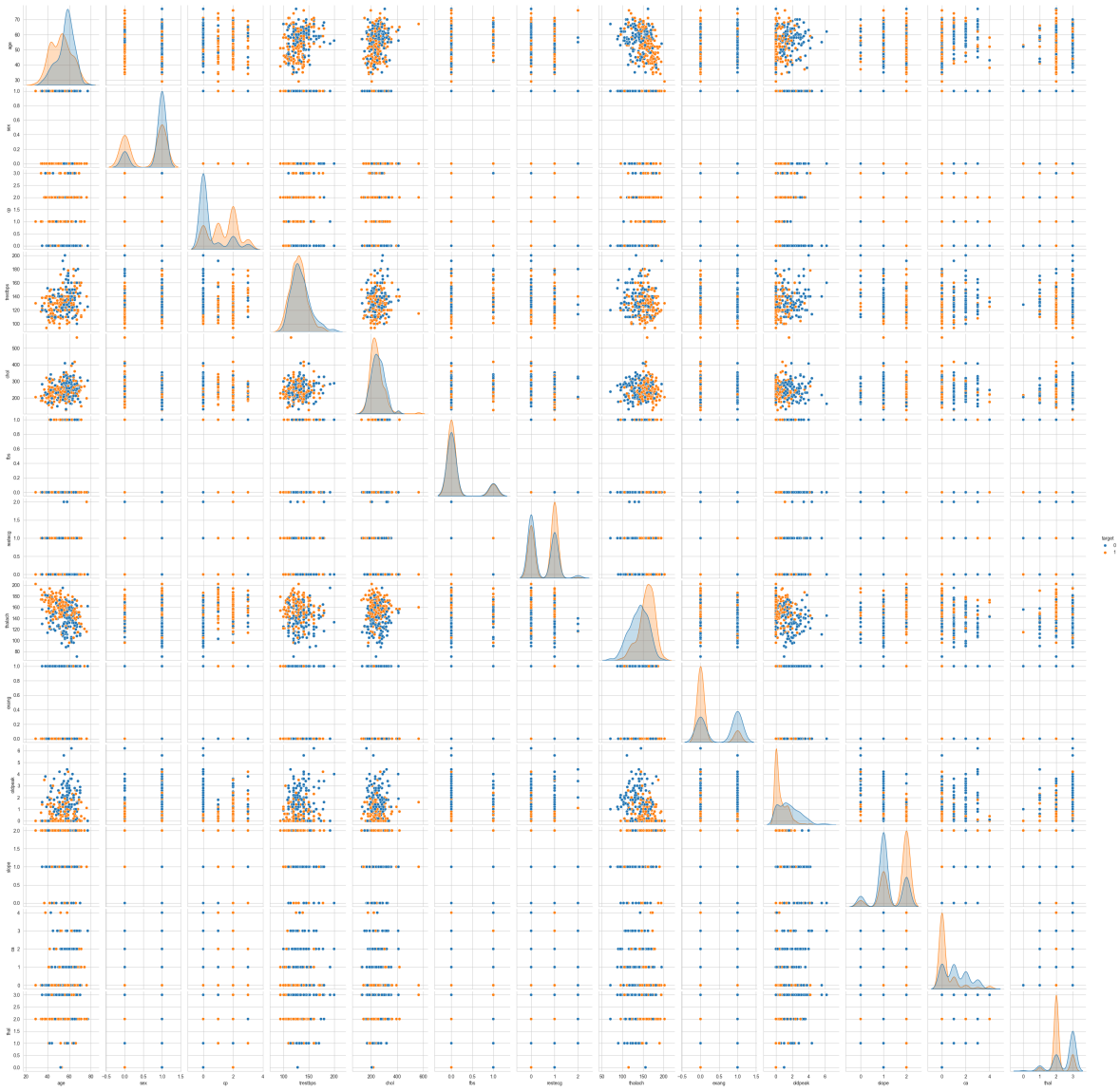
```
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52    1   0       125   212    0        1      168      0      1.0      2
1   53    1   0       140   203    1        0      155      1      3.1      0
2   70    1   0       145   174    0        1      125      1      2.6      0
3   61    1   0       148   203    0        1      161      0      0.0      2
4   62    0   0       138   294    1        1      106      0      1.9      1

   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0

Dataset shape: (1025, 14)

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1025 non-null   int64
 1   sex       1025 non-null   int64
 2   cp        1025 non-null   int64
 3   trestbps  1025 non-null   int64
 4   chol      1025 non-null   int64
 5   fbs       1025 non-null   int64
 6   restecg   1025 non-null   int64
 7   thalach   1025 non-null   int64
 8   exang     1025 non-null   int64
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64
 11  ca        1025 non-null   int64
 12  thal      1025 non-null   int64
 13  target    1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
None

Statistical Summary:
               age          sex           cp     trestbps        chol  \
count  1025.000000  1025.000000  1025.000000  1025.000000  1025.00000
mean     54.434146     0.695610     0.942439   131.611707   246.00000
std       9.072290     0.460373     1.029641    17.516718    51.59251
min      29.000000     0.000000     0.000000    94.000000   126.00000
25%      48.000000     0.000000     0.000000   120.000000   211.00000
50%      56.000000     1.000000     1.000000   130.000000   240.00000
75%      61.000000     1.000000     2.000000   140.000000   275.00000
max      77.000000     1.000000     3.000000   200.000000   564.00000

               fbs      restecg      thalach        exang      oldpeak  \
count  1025.000000  1025.000000  1025.000000  1025.000000  1025.000000
mean      0.149268     0.529756   149.114146     0.336585     1.071512
std       0.356527     0.527878    23.005724     0.472772     1.175053
min       0.000000     0.000000    71.000000     0.000000     0.000000
25%       0.000000     0.000000   132.000000     0.000000     0.000000
50%       0.000000     1.000000   152.000000     0.000000     0.800000
75%       0.000000     1.000000   166.000000     1.000000     1.800000
max       1.000000     2.000000   202.000000     1.000000     6.200000
```

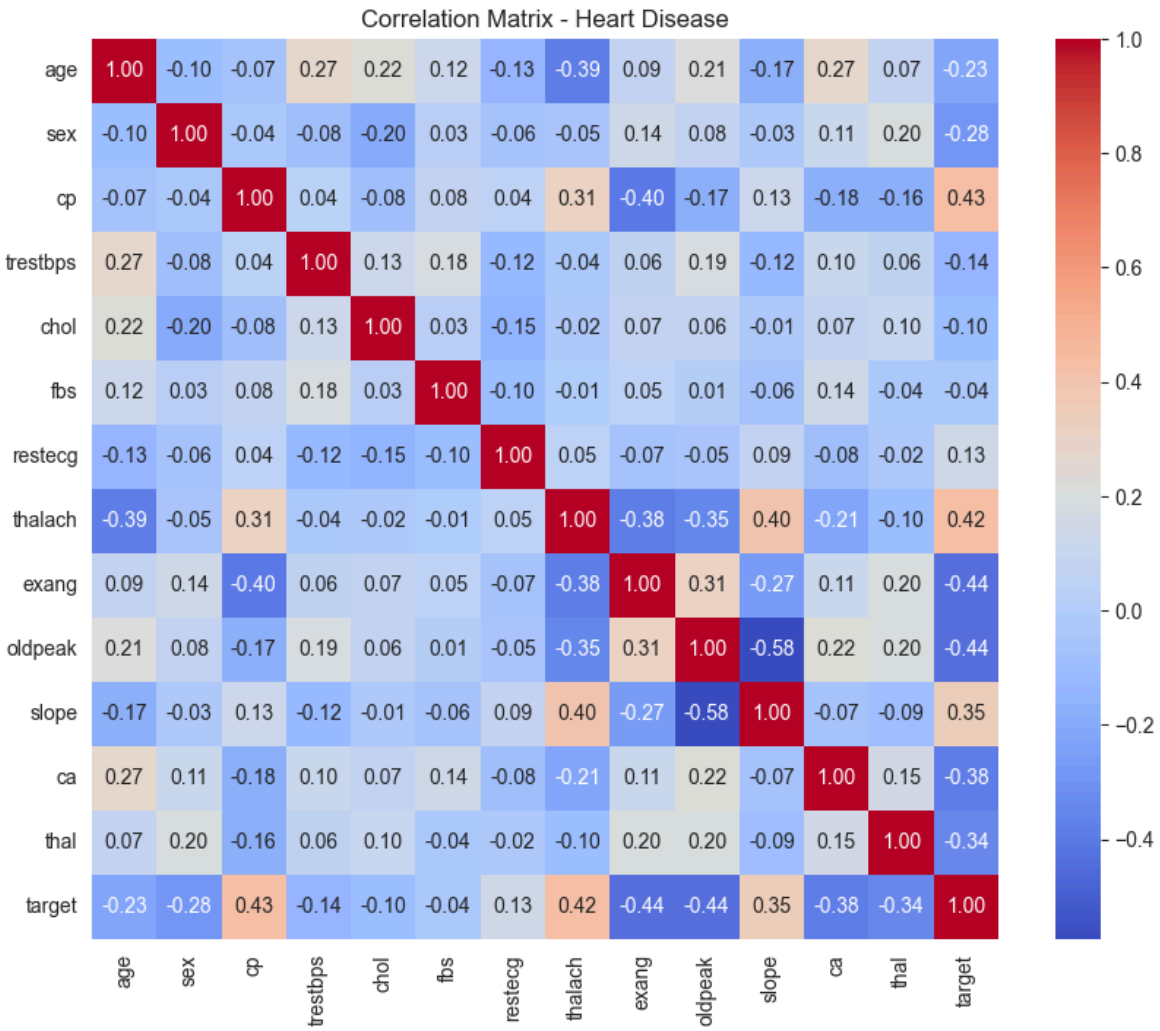
	slope	ca	thal	target
count	1025.000000	1025.000000	1025.000000	1025.000000
mean	1.385366	0.754146	2.323902	0.513171
std	0.617755	1.030798	0.620660	0.500070
min	0.000000	0.000000	0.000000	0.000000
25%	1.000000	0.000000	2.000000	0.000000
50%	1.000000	0.000000	2.000000	1.000000
75%	2.000000	1.000000	3.000000	1.000000
max	2.000000	4.000000	3.000000	1.000000

Missing values:

age	0
sex	0
cp	0
trestbps	0
chol	0
fbs	0
restecg	0
thalach	0
exang	0
oldpeak	0
slope	0
ca	0
thal	0
target	0

dtype: int64


age
sex
cp
trestbps
chol
fbs
restecg
thalach
exang
oldpeak
slope
ca
thal
target
0
1


Correlation Matrix - Heart Disease
age 1.00 -0.10 -0.07 0.27 0.22 0.12 -0.13 -0.39 0.09 0.21 -0.17 0.27 0.07 -0.23
sex -0.10 1.00 -0.04 -0.08 -0.20 0.03 -0.06 -0.05 0.14 0.08 -0.03 0.11 0.20 -0.28
cp -0.07 -0.04 1.00 0.04 -0.08 0.08 0.04 0.31 -0.40 -0.17 0.13 -0.18 -0.16 0.43
trestbps 0.27 -0.08 0.04 1.00 0.13 0.18 -0.12 -0.04 0.06 0.19 -0.12 0.10 0.06 -0.14
chol 0.22 -0.20 -0.08 0.13 1.00 0.03 -0.15 -0.02 0.07 0.06 -0.01 0.07 0.10 -0.10
fbs 0.12 0.03 0.08 0.18 0.03 1.00 -0.10 -0.01 0.05 0.01 -0.06 0.14 -0.04 -0.04
restecg -0.13 -0.06 0.04 -0.12 -0.15 -0.10 1.00 0.05 -0.07 -0.05 0.09 -0.08 -0.02 0.13
thalach -0.39 -0.05 0.31 -0.04 -0.02 -0.01 0.05 1.00 -0.38 -0.35 0.40 -0.21 -0.10 0.42
exang 0.09 0.14 -0.40 0.06 0.07 0.05 -0.07 -0.38 1.00 0.31 -0.27 0.11 0.20 -0.44
oldpeak 0.21 0.08 -0.17 0.19 0.06 0.01 -0.05 -0.35 0.31 1.00 -0.58 0.22 0.20 -0.44
slope -0.17 -0.03 0.13 -0.12 -0.01 -0.06 0.09 0.40 -0.27 -0.58 1.00 -0.07 -0.09 0.35
ca 0.27 0.11 -0.18 0.10 0.07 0.14 -0.08 -0.21 0.11 0.22 -0.07 1.00 0.15 -0.38
thal 0.07 0.20 -0.16 0.06 0.10 -0.04 -0.02 -0.10 0.20 0.20 -0.09 0.15 1.00 -0.34
target -0.23 -0.28 0.43 -0.14 -0.10 -0.04 0.13 0.42 -0.44 -0.44 0.35 -0.38 -0.34 1.00
age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal target
1.0 0.8 0.6 0.4 0.2 0.0 −0.2 −0.4

```
Decision Tree Accuracy: 0.9902597402597403
Confusion Matrix:
 [[159   0]
 [  3 146]]
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       159
           1       1.00      0.98      0.99       149

    accuracy                           0.99       308
   macro avg       0.99      0.99      0.99       308
weighted avg       0.99      0.99      0.99       308


KNN Accuracy: 0.8409090909090909
Confusion Matrix:
 [[130  29]
 [ 20 129]]
              precision    recall  f1-score   support

           0       0.87      0.82      0.84       159
           1       0.82      0.87      0.84       149

    accuracy                           0.84       308
   macro avg       0.84      0.84      0.84       308
weighted avg       0.84      0.84      0.84       308


Naive Bayes Accuracy: 0.814935064935065
Confusion Matrix:
 [[118  41]
 [ 16 133]]
              precision    recall  f1-score   support

           0       0.88      0.74      0.81       159
           1       0.76      0.89      0.82       149

    accuracy                           0.81       308
   macro avg       0.82      0.82      0.81       308
weighted avg       0.82      0.81      0.81       308


SVM Accuracy: 0.8084415584415584
Confusion Matrix:
 [[119  40]
 [ 19 130]]
              precision    recall  f1-score   support

           0       0.86      0.75      0.80       159
           1       0.76      0.87      0.82       149

    accuracy                           0.81       308
   macro avg       0.81      0.81      0.81       308
weighted avg       0.82      0.81      0.81       308
```

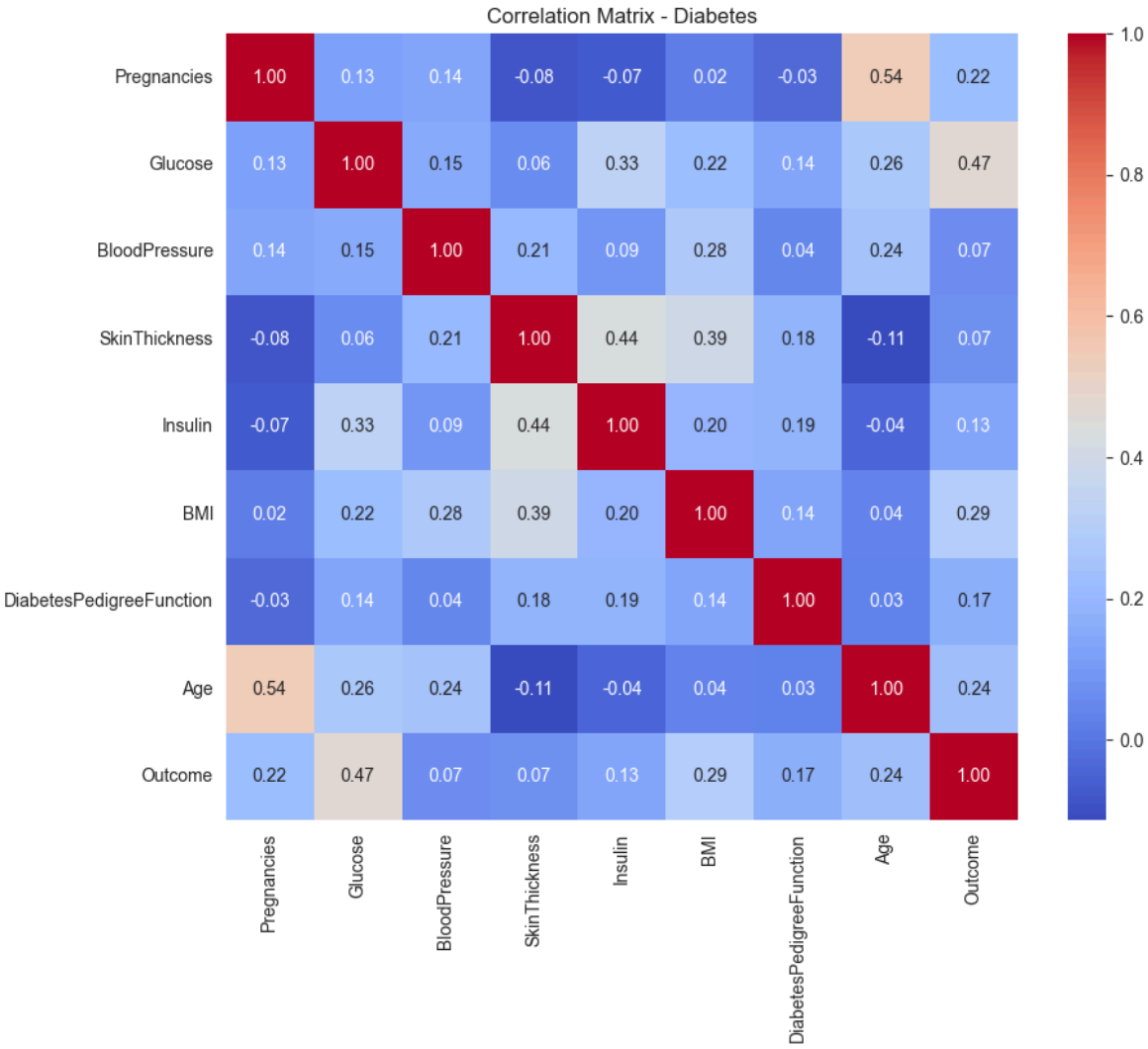
In [4]:
```
#PIMA INDIANS DIABETES DATASET
# Load dataset (Kaggle: uciml/pima-indians-diabetes-database)
diabetes = pd.read_csv('diabetes_ds.csv')
```

```
print(diabetes.head())
print("\nDataset shape:", diabetes.shape)
print("\nDataset Info:")
print(diabetes.info())
print("\nStatistical Summary:")
print(diabetes.describe())

# Missing values
print("\nMissing values:")
print(diabetes.isnull().sum())

# Pairplot
sns.pairplot(diabetes, hue="Outcome")
plt.show()

# Correlation heatmap
plt.figure(figsize=(10,8))
sns.heatmap(diabetes.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Matrix - Diabetes")
plt.show()

# Prepare data
X = diabetes.drop('Outcome', axis=1)
y = diabetes['Outcome']

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_

# Scale
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Decision Tree
dt = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
print("\nDecision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)
print("\nKNN Accuracy:", accuracy_score(y_test, y_pred_knn))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))

# Naive Bayes
nb = GaussianNB()
nb.fit(X_train, y_train)
y_pred_nb = nb.predict(X_test)
print("\nNaive Bayes Accuracy:", accuracy_score(y_test, y_pred_nb))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_nb))
print(classification_report(y_test, y_pred_nb))

# SVM
svm = SVC(kernel='linear', random_state=42)
svm.fit(X_train_scaled, y_train)
```

```
y_pred_svm = svm.predict(X_test_scaled)
print("\nSVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Dataset shape: (768, 9)

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None

Statistical Summary:
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.992578                  0.471876   33.240885    0.348958
std      7.884160                  0.331329   11.760232    0.476951
min      0.000000                  0.078000   21.000000    0.000000
25%     27.300000                  0.243750   24.000000    0.000000
50%     32.000000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000

Missing values:
Pregnancies                 0
Glucose                     0
BloodPressure               0
```

```
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```


Correlation Matrix - Diabetes
Pregnancies 1.00 0.13 0.14 -0.08 -0.07 0.02 -0.03 0.54 0.22
Glucose 0.13 1.00 0.15 0.06 0.33 0.22 0.14 0.26 0.47
BloodPressure 0.14 0.15 1.00 0.21 0.09 0.28 0.04 0.24 0.07
SkinThickness -0.08 0.06 0.21 1.00 0.44 0.39 0.18 -0.11 0.07
Insulin -0.07 0.33 0.09 0.44 1.00 0.20 0.19 -0.04 0.13
BMI 0.02 0.22 0.28 0.39 0.20 1.00 0.14 0.04 0.29
DiabetesPedigreeFunction -0.03 0.14 0.04 0.18 0.19 0.14 1.00 0.03 0.17
Age 0.54 0.26 0.24 -0.11 -0.04 0.04 0.03 1.00 0.24
Outcome 0.22 0.47 0.07 0.07 0.13 0.29 0.17 0.24 1.00
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome
1.0
0.8
0.6
0.4
0.2
0.0

```
Decision Tree Accuracy: 0.7272727272727273
Confusion Matrix:
 [[118  33]
 [ 30  50]]
              precision    recall  f1-score   support

           0       0.80      0.78      0.79       151
           1       0.60      0.62      0.61        80

    accuracy                           0.73       231
   macro avg       0.70      0.70      0.70       231
weighted avg       0.73      0.73      0.73       231


KNN Accuracy: 0.7012987012987013
Confusion Matrix:
 [[119  32]
 [ 37  43]]
              precision    recall  f1-score   support

           0       0.76      0.79      0.78       151
           1       0.57      0.54      0.55        80

    accuracy                           0.70       231
   macro avg       0.67      0.66      0.67       231
weighted avg       0.70      0.70      0.70       231


Naive Bayes Accuracy: 0.7445887445887446
Confusion Matrix:
 [[119  32]
 [ 27  53]]
              precision    recall  f1-score   support

           0       0.82      0.79      0.80       151
           1       0.62      0.66      0.64        80

    accuracy                           0.74       231
   macro avg       0.72      0.73      0.72       231
weighted avg       0.75      0.74      0.75       231


SVM Accuracy: 0.7489177489177489
Confusion Matrix:
 [[123  28]
 [ 30  50]]
              precision    recall  f1-score   support

           0       0.80      0.81      0.81       151
           1       0.64      0.62      0.63        80

    accuracy                           0.75       231
   macro avg       0.72      0.72      0.72       231
weighted avg       0.75      0.75      0.75       231
```

In [ ]:

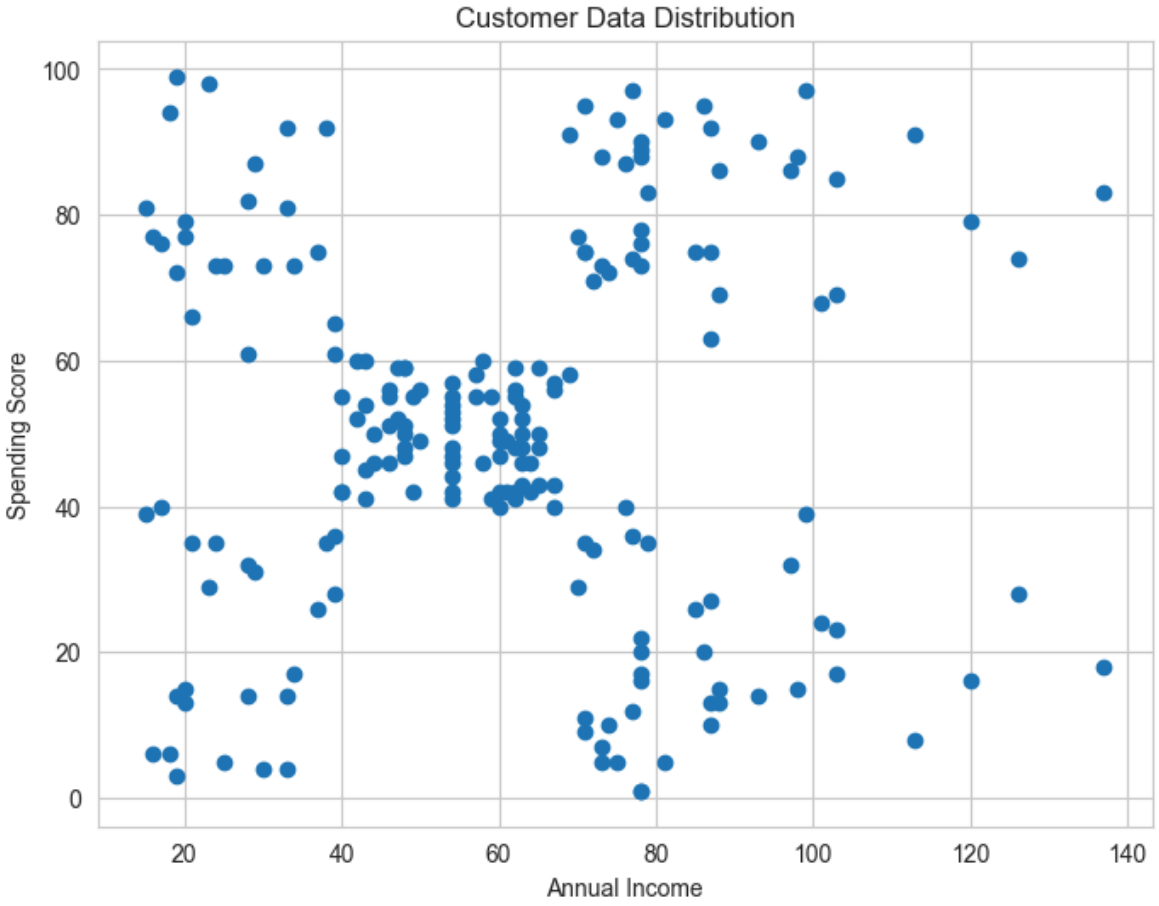
In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans, AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

sns.set_style("whitegrid")
```

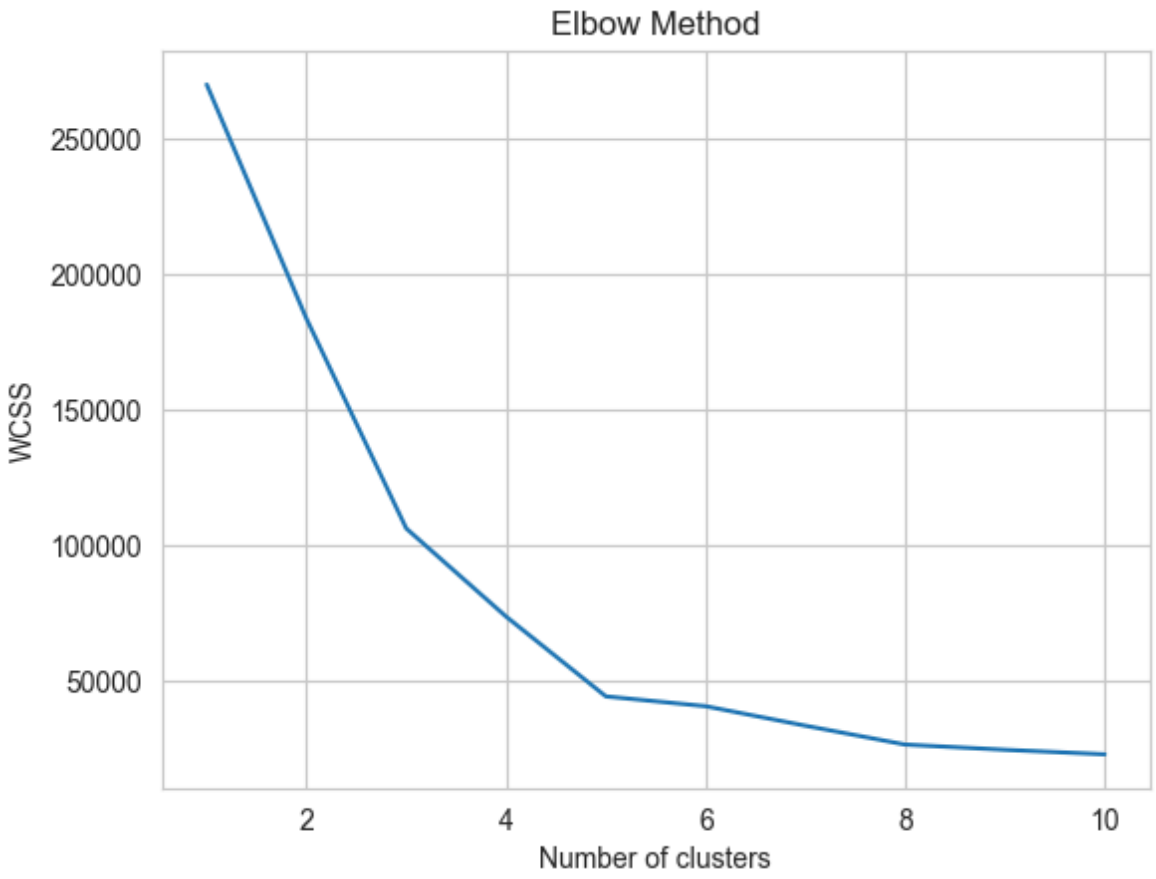
In [ ]:
```
#K-Means & Hierarchical Clustering

#MALL CUSTOMERS DATASET
df = pd.read_csv('Mall_Customers.csv') # Kaggle link https://www.kaggle.com/data
print(df.head(), "\n\n")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

df.dropna(inplace=True)
data = df[['Annual Income (k$)','Spending Score (1-100)']]
```

```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40


<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    str
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), str(1)
memory usage: 7.9 KB
None

Missing Values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
```

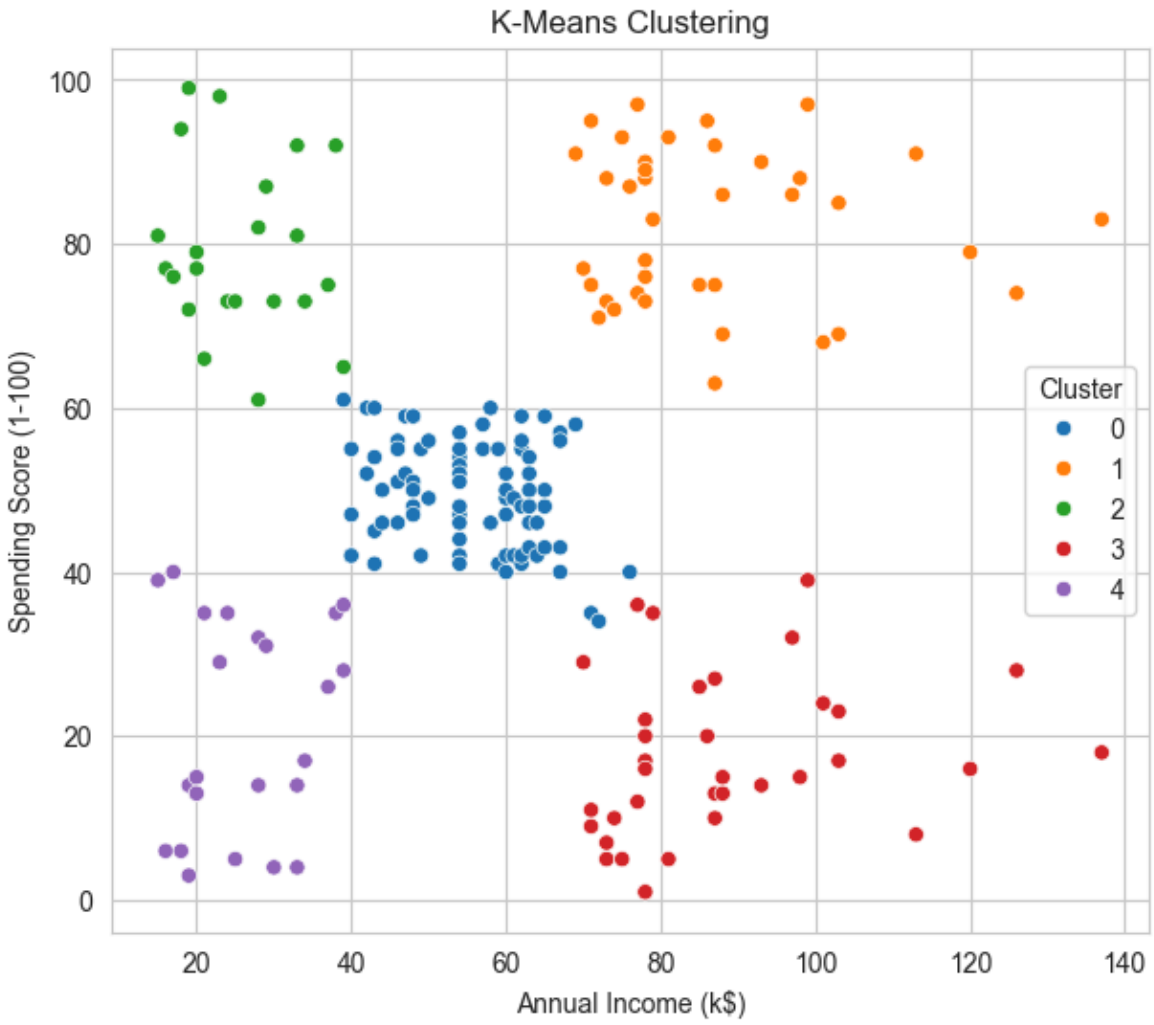
In [ ]:
```
plt.figure(figsize=(8,6))
plt.scatter(data['Annual Income (k$)'], data['Spending Score (1-100)'])
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Customer Data Distribution")
plt.show()
```

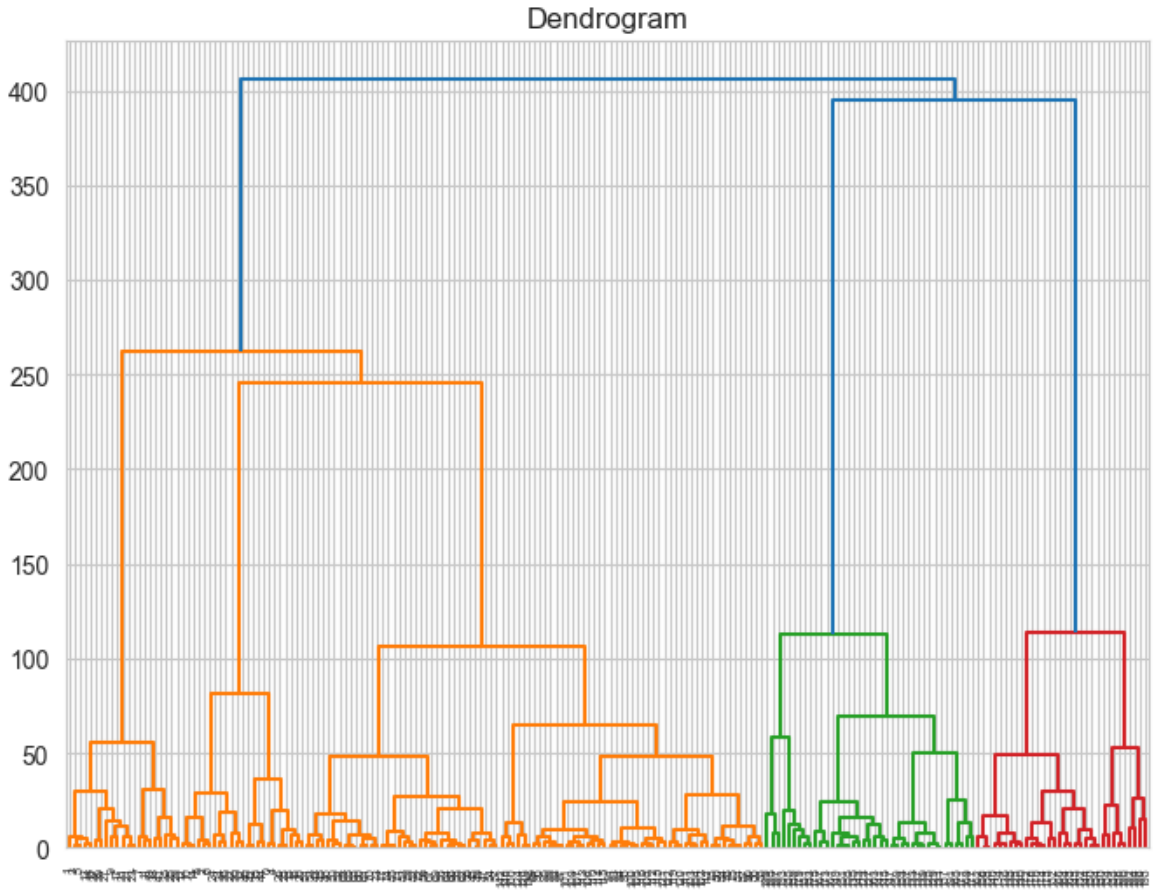
```
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)

plt.plot(range(1,11), wcss)
plt.title("Elbow Method")
plt.xlabel("Number of clusters")
plt.ylabel("WCSS")
plt.show()
```

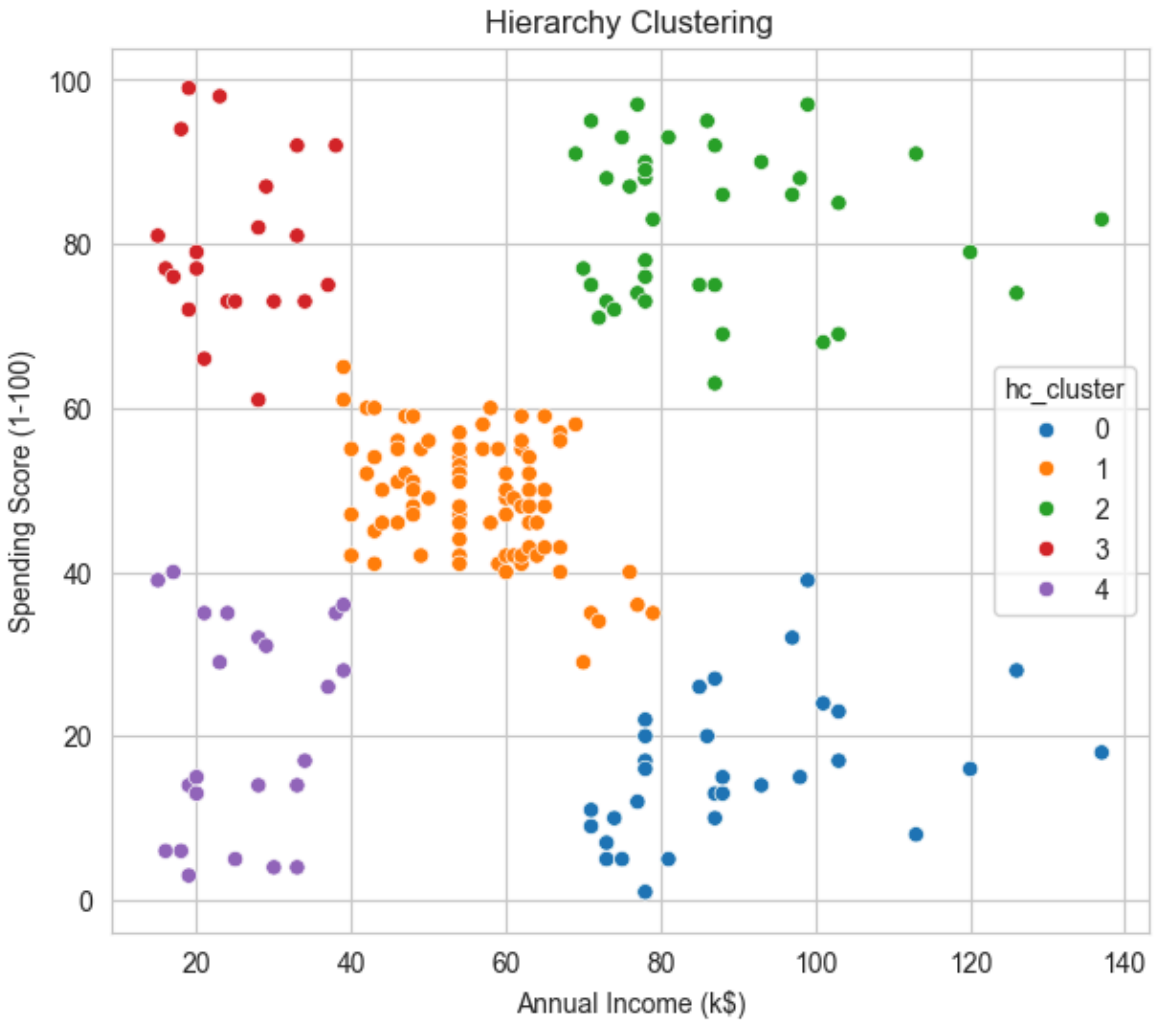




In [5]:
```
# K-Means Clustering
kmeans = KMeans(n_clusters=5, random_state=42)
clusters = kmeans.fit_predict(data)
data["Cluster"] = clusters

plt.figure(figsize=(7,6))
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)', hue="Cluster
plt.title("K-Means Clustering")
plt.show()

# Hierarchical Clustering - Dendrogram (exact from PDF)
linked = linkage(data[['Annual Income (k$)','Spending Score (1-100)']], method='
plt.figure(figsize=(8,6))
dendrogram(linked)
plt.title("Dendrogram")
plt.show()

# Agglomerative Hierarchical Clustering (exact from PDF)
hc = AgglomerativeClustering(n_clusters=5)
data['hc_cluster'] = hc.fit_predict(data[['Annual Income (k$)','Spending Score (

plt.figure(figsize=(7,6))
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)', hue='hc_clus
plt.title("Hierarchy Clustering")
plt.show()
```


K-Means Clustering
Spending Score (1-100)
Annual Income (k$)
Cluster
0
1
2
3
4
0
20
40
60
80
100
20
40
60
80
100
120
140


Dendrogram
0
50
100
150
200
250
300
350
400

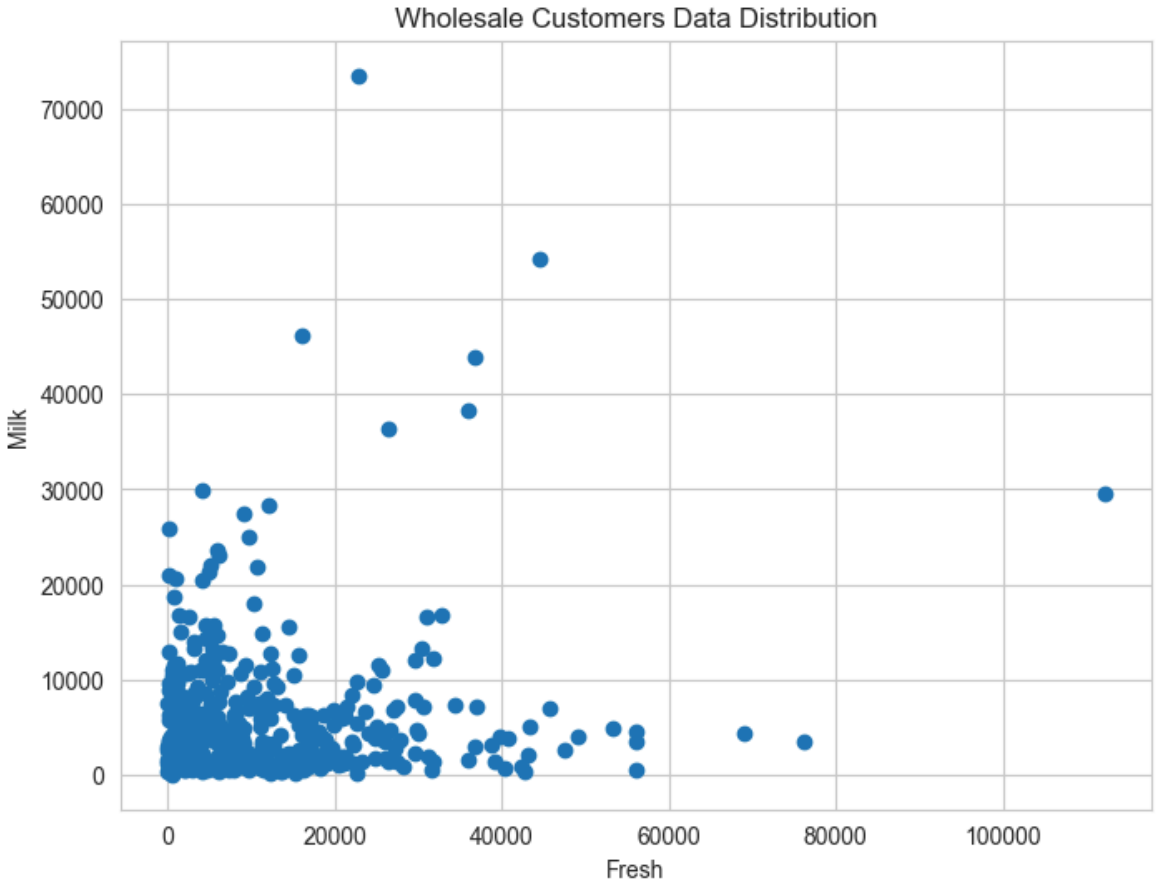


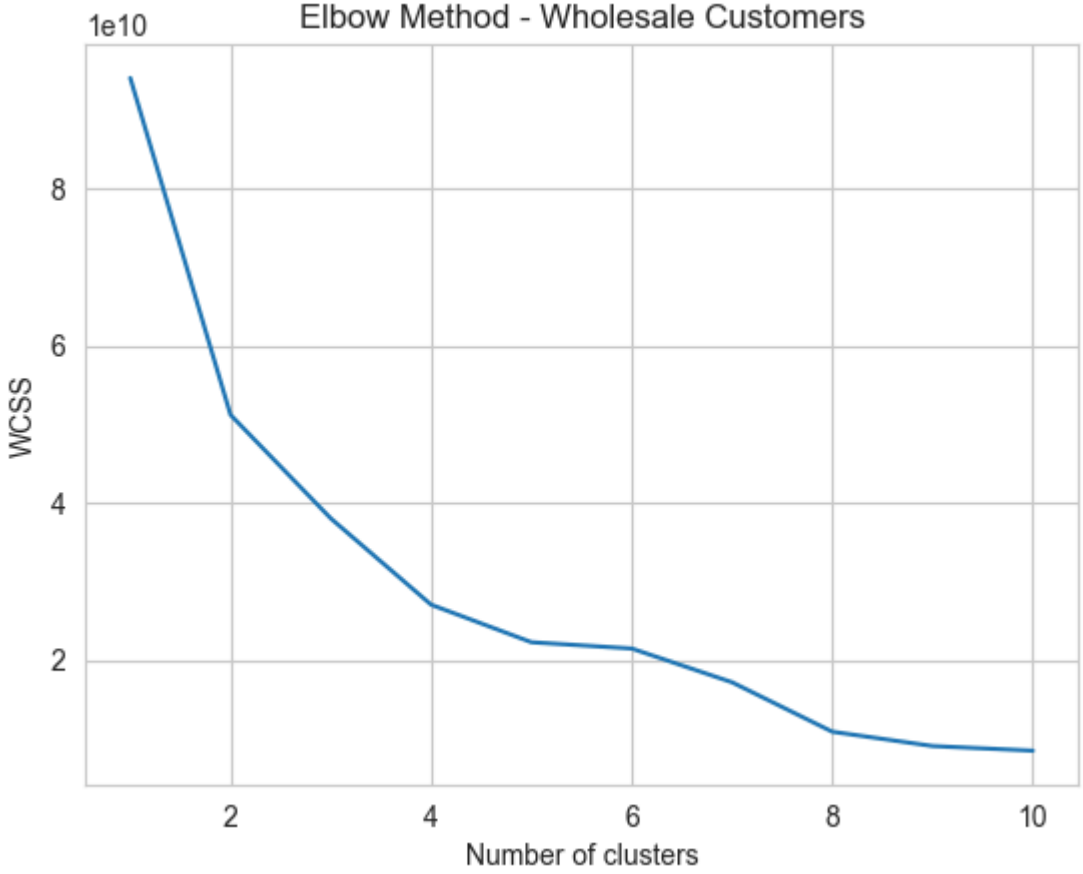
In [7]:
```
#WHOLESALE CUSTOMERS DATASET
wholesale = pd.read_csv('Wholesale_customers_data.csv')
print(wholesale.head(), "\n\n")
print(wholesale.info())

print("\nMissing Values:")
print(wholesale.isnull().sum())

# Using two features (mirroring Mall_Customers)
data2 = wholesale[['Fresh', 'Milk']]
print("\nSelected features for clustering: Fresh & Milk")
```

```
   Channel  Region  Fresh  Milk  Grocery  Frozen  Detergents_Paper  Delicassen
0        2       3  12669  9656     7561     214              2674        1338
1        2       3   7057  9810     9568    1762              3293        1776
2        2       3   6353  8808     7684    2405              3516        7844
3        1       3  13265  1196     4221    6404               507        1788
4        2       3  22615  5410     7198    3915              1777        5185
```

```
<class 'pandas.DataFrame'>
RangeIndex: 440 entries, 0 to 439
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Channel           440 non-null    int64
 1   Region            440 non-null    int64
 2   Fresh             440 non-null    int64
 3   Milk              440 non-null    int64
 4   Grocery           440 non-null    int64
 5   Frozen            440 non-null    int64
 6   Detergents_Paper  440 non-null    int64
 7   Delicassen        440 non-null    int64
dtypes: int64(8)
memory usage: 27.6 KB
None

Missing Values:
Channel             0
Region              0
Fresh               0
Milk                0
Grocery             0
Frozen              0
Detergents_Paper    0
Delicassen          0
dtype: int64

Selected features for clustering: Fresh & Milk
```

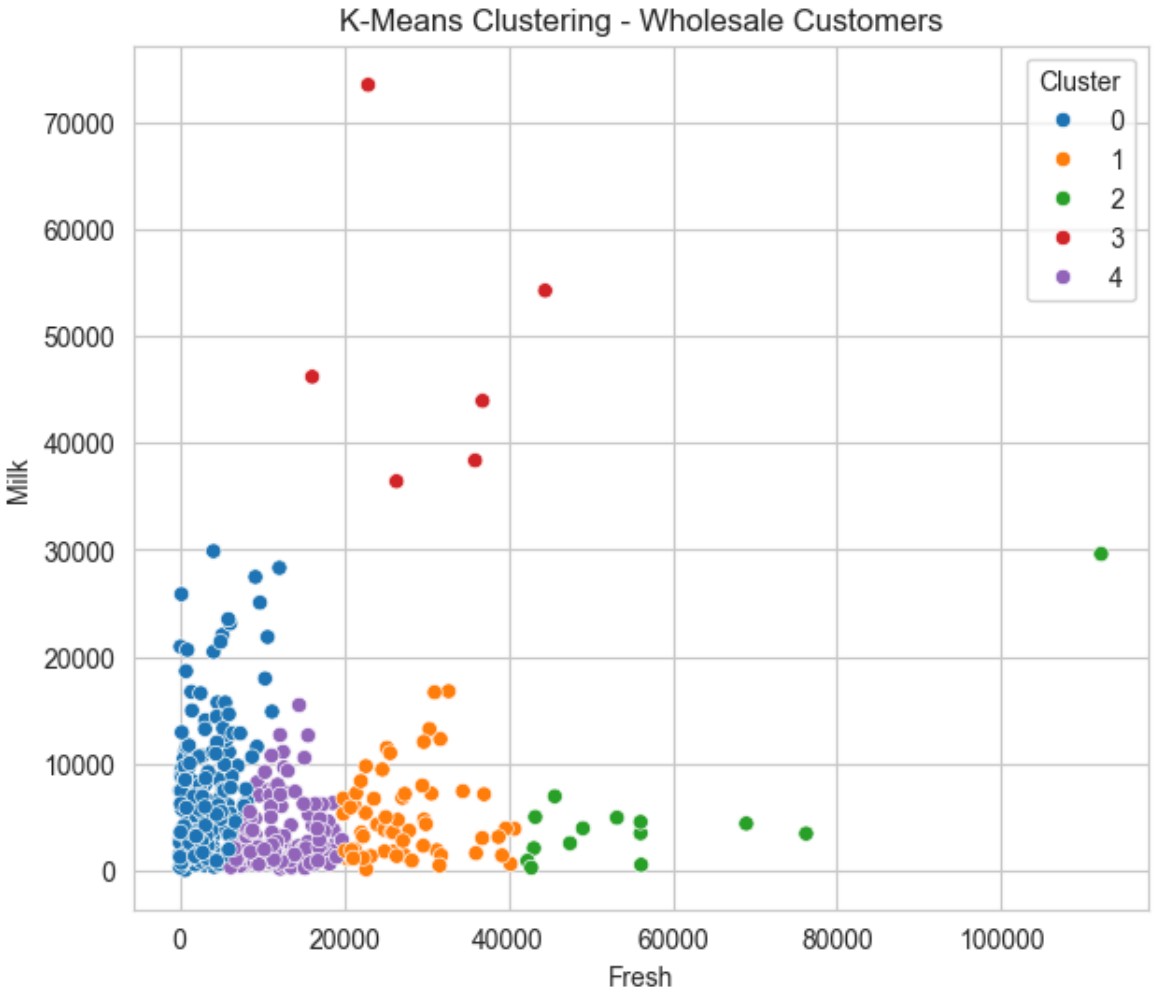
In [8]:
```
# Scatter plot
plt.figure(figsize=(8,6))
plt.scatter(data2['Fresh'], data2['Milk'])
plt.xlabel("Fresh")
plt.ylabel("Milk")
plt.title("Wholesale Customers Data Distribution")
plt.show()

# Elbow Method
wcss2 = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(data2)
    wcss2.append(kmeans.inertia_)

plt.plot(range(1,11), wcss2)
plt.title("Elbow Method - Wholesale Customers")
plt.xlabel("Number of clusters")
plt.ylabel("WCSS")
plt.show()
```

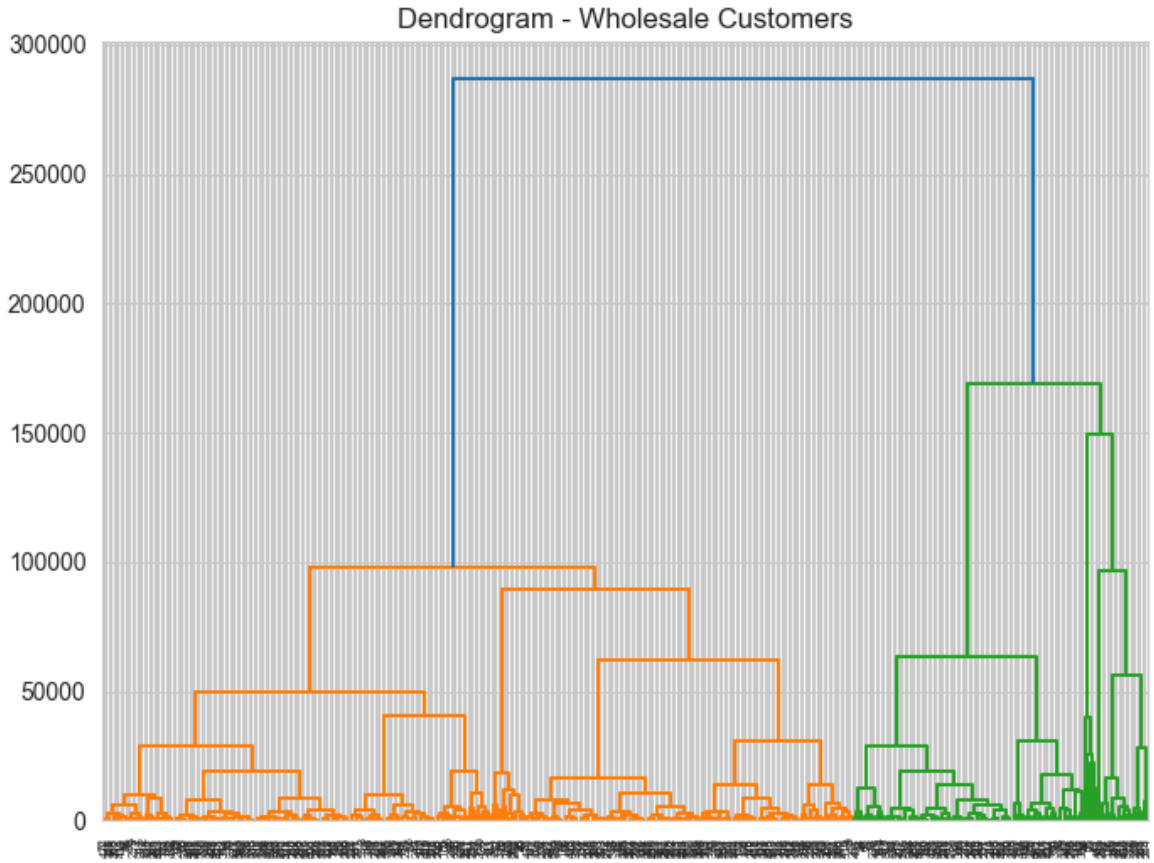


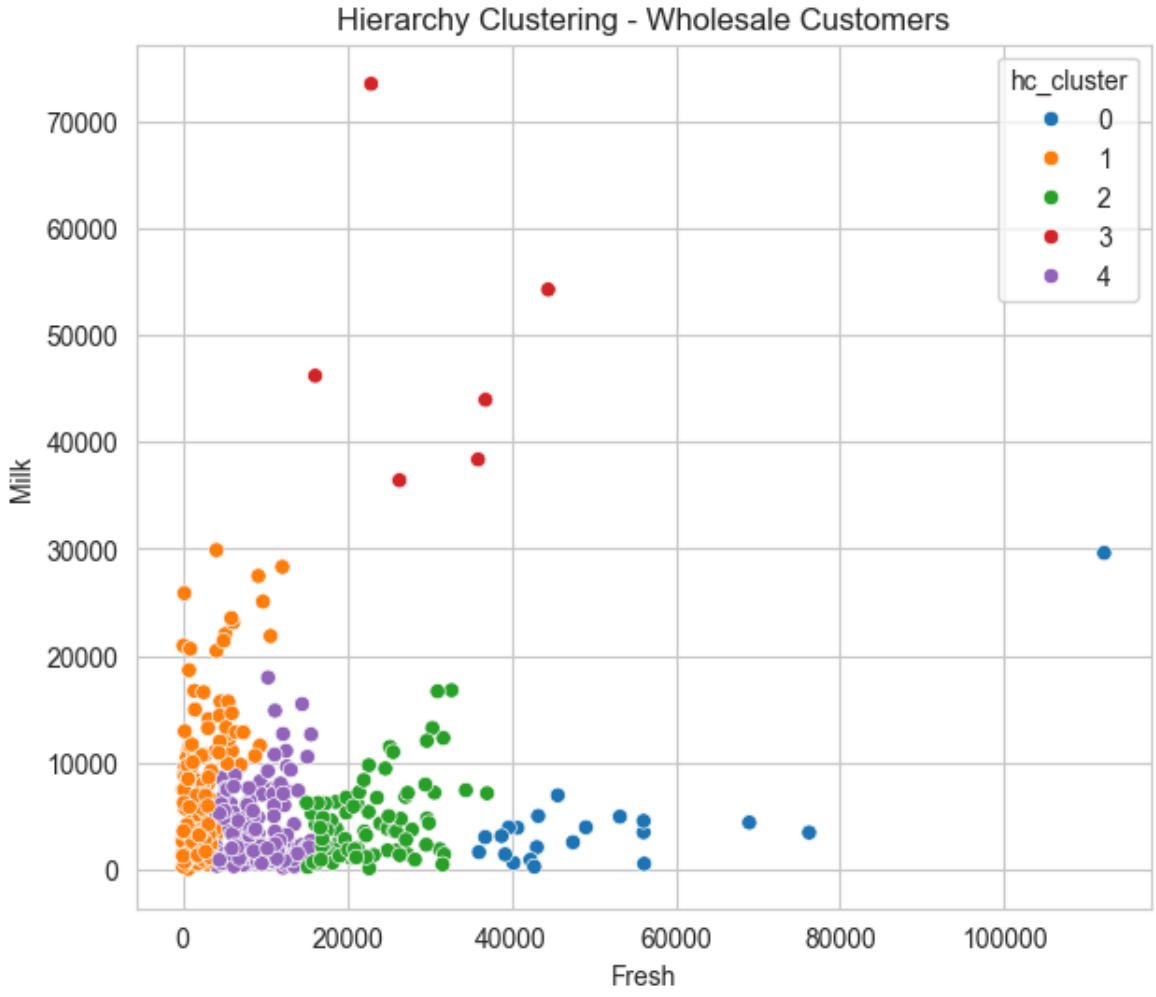


In [10]:
```
# K-Means Clustering
kmeans2 = KMeans(n_clusters=5, random_state=42)
clusters2 = kmeans2.fit_predict(data2)
data2["Cluster"] = clusters2
```

```
plt.figure(figsize=(7,6))
sns.scatterplot(x='Fresh', y='Milk', hue="Cluster", data=data2, palette='tab10')
plt.title("K-Means Clustering - Wholesale Customers")
plt.show()

# Hierarchical - Dendrogram
linked2 = linkage(data2[['Fresh', 'Milk']], method='ward')
plt.figure(figsize=(8,6))
dendrogram(linked2)
plt.title("Dendrogram - Wholesale Customers")
plt.show()

# Agglomerative Hierarchical
hc2 = AgglomerativeClustering(n_clusters=5)
data2['hc_cluster'] = hc2.fit_predict(data2[['Fresh', 'Milk']])

plt.figure(figsize=(7,6))
sns.scatterplot(x='Fresh', y='Milk', hue='hc_cluster', data=data2, palette='tab1
plt.title("Hierarchy Clustering - Wholesale Customers")
plt.show()
```




Dendrogram - Wholesale Customers
300000
250000
200000
150000
100000
50000
0


Hierarchy Clustering - Wholesale Customers
hc_cluster
0
1
2
3
4
Milk
70000
60000
50000
40000
30000
20000
10000
0
0
20000
40000
60000
80000
100000
Fresh

In [1]:
```
#Ensemble Techniques (Bagging, Random Forest, Boosting)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier, AdaBoost
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
```

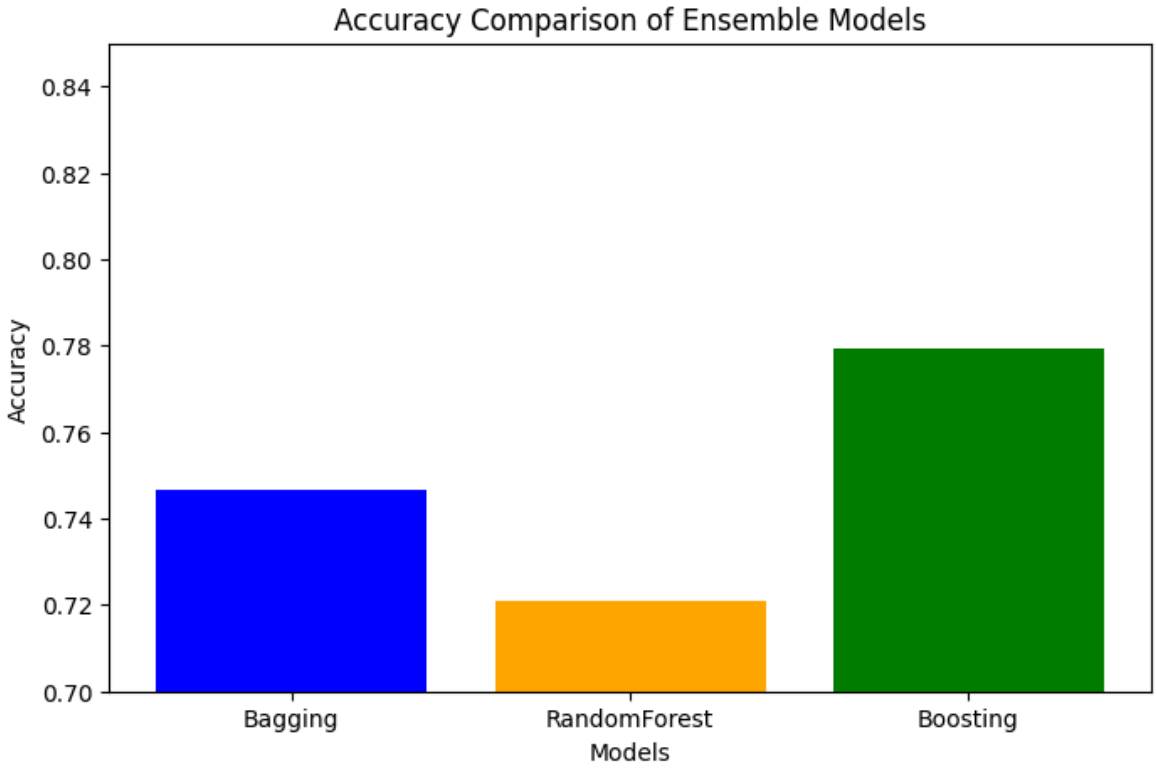
In [3]:
```
#Same diabetes dataset from Kaggle (https://www.kaggle.com/datasets/uciml/pima-i
data = pd.read_csv('diabetes_ds.csv')
print("Dataset Shape:", data.shape)
print(data.head())

X = data.drop("Outcome", axis=1)
y = data["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

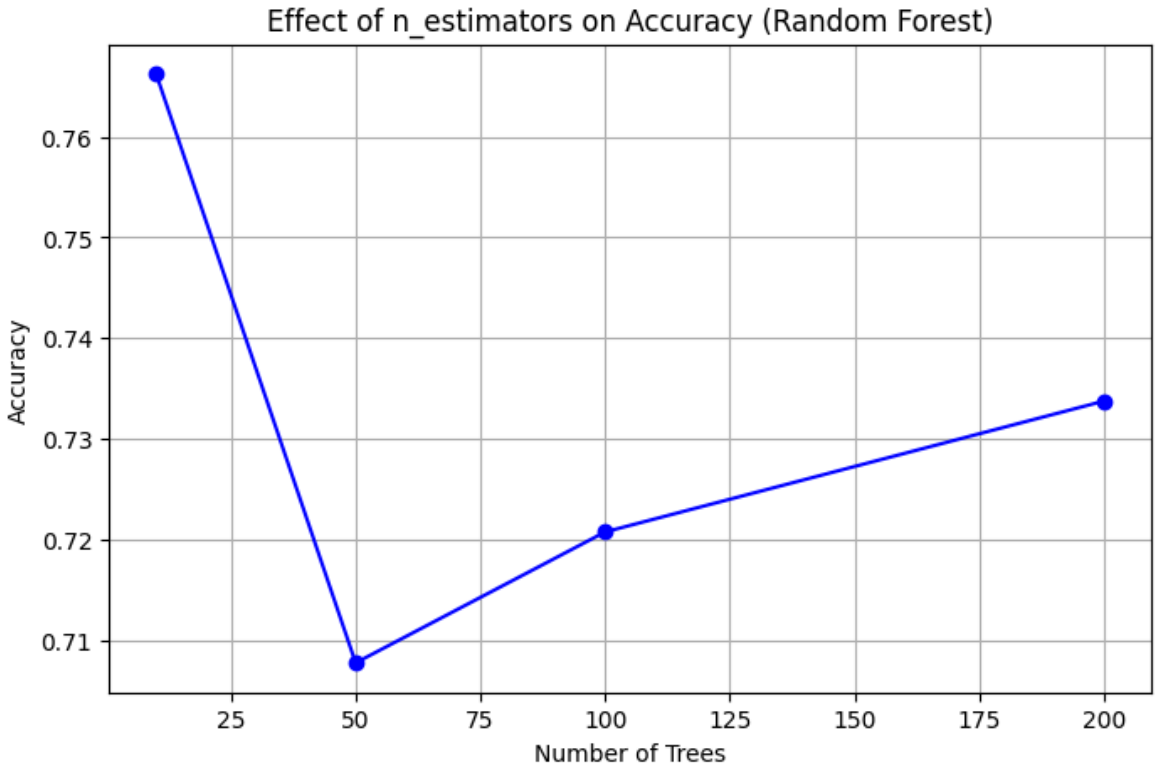
```
Dataset Shape: (768, 9)
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

In [4]:
```
# Bagging Classifier
bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    random_state=42
)
bagging.fit(X_train, y_train)
y_pred_bagg = bagging.predict(X_test)
print("Bagging accuracy :", accuracy_score(y_test, y_pred_bagg))

# Random Forest (exact from your program6.pdf)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
print("Random Forest accuracy :", accuracy_score(y_test, y_pred_rf))

# AdaBoost (exact from your program6.pdf)
boost = AdaBoostClassifier(n_estimators=50, random_state=42)
boost.fit(X_train, y_train)
y_pred_boos = boost.predict(X_test)
print("Boosting accuracy :", accuracy_score(y_test, y_pred_boos))
```

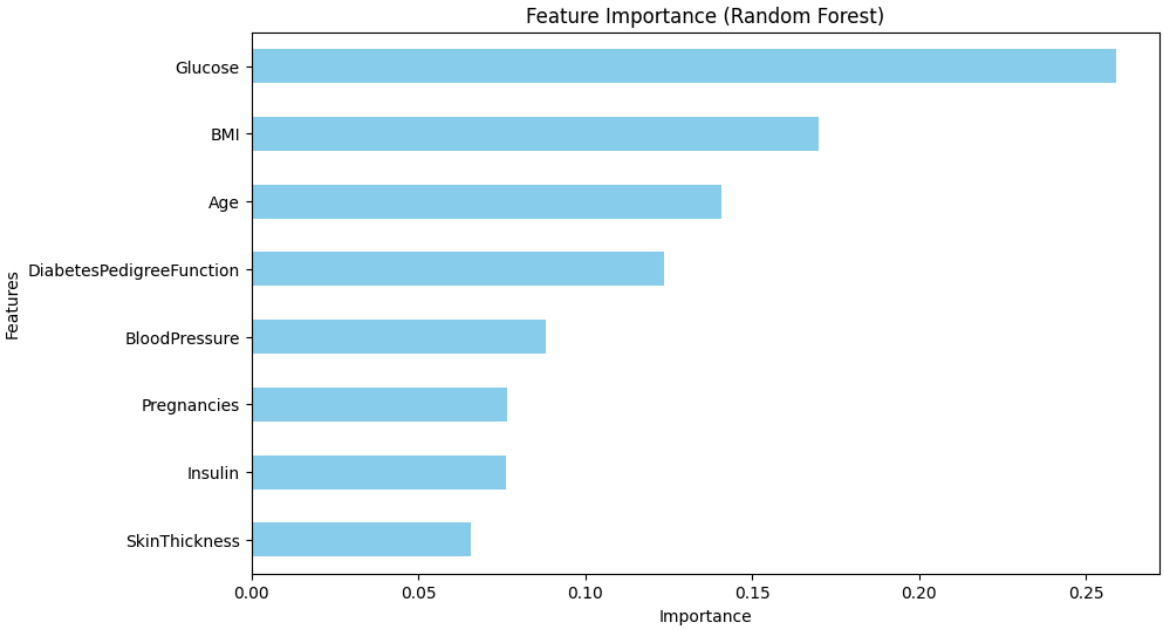
```
Bagging accuracy : 0.7467532467532467
Random Forest accuracy : 0.7207792207792207
Boosting accuracy : 0.7792207792207793
```

In [5]:
```python
#comparison of accuracies
bag_acc = accuracy_score(y_test, y_pred_bagg)
rf_acc = accuracy_score(y_test, y_pred_rf)
boo_acc = accuracy_score(y_test, y_pred_boos)

models = ['Bagging', 'RandomForest', 'Boosting']
accuracies = [bag_acc, rf_acc, boo_acc]

plt.figure(figsize=(8,5))
plt.bar(models, accuracies, color=['blue', 'orange', 'green'])
plt.title("Accuracy Comparison of Ensemble Models")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.ylim(0.7, 0.85)
plt.show()

n_values = [10, 50, 100, 200]
rf_scores = []
for n in n_values:
    model = RandomForestClassifier(n_estimators=n, random_state=42)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    rf_scores.append(accuracy_score(y_test, pred))

plt.figure(figsize=(8,5))
plt.plot(n_values, rf_scores, marker='o', color='blue')
plt.title("Effect of n_estimators on Accuracy (Random Forest)")
plt.xlabel("Number of Trees")
plt.ylabel("Accuracy")
plt.grid(True)
plt.show()
```





In [6]:
```
# Feature importance
importances = rf.feature_importances_
features = X.columns
fea_imp = pd.Series(importances, index=features)

plt.figure(figsize=(10,6))
fea_imp.sort_values().plot(kind='barh', color='skyblue')
plt.title("Feature Importance (Random Forest)")
plt.xlabel("Importance")
plt.ylabel("Features")
plt.show()
```



In [7]:
```
# LEARNER SVM & Naive Bayes as base

# Bagging
bagging_svm = BaggingClassifier(
```

```
    estimator=SVC(),
    n_estimators=50,
    random_state=42
)
bagging_svm.fit(X_train, y_train)
y_pred_bagg_svm = bagging_svm.predict(X_test)
print("Bagging (SVM base) Accuracy:", accuracy_score(y_test, y_pred_bagg_svm))

# Bagging with Naive Bayes as base learner
bagging_nb = BaggingClassifier(
    estimator=GaussianNB(),
    n_estimators=50,
    random_state=42
)
bagging_nb.fit(X_train, y_train)
y_pred_bagg_nb = bagging_nb.predict(X_test)
print("Bagging (Naive Bayes base) Accuracy:", accuracy_score(y_test, y_pred_bagg
```

```
Bagging (SVM base) Accuracy: 0.7662337662337663
Bagging (Naive Bayes base) Accuracy: 0.7792207792207793
```

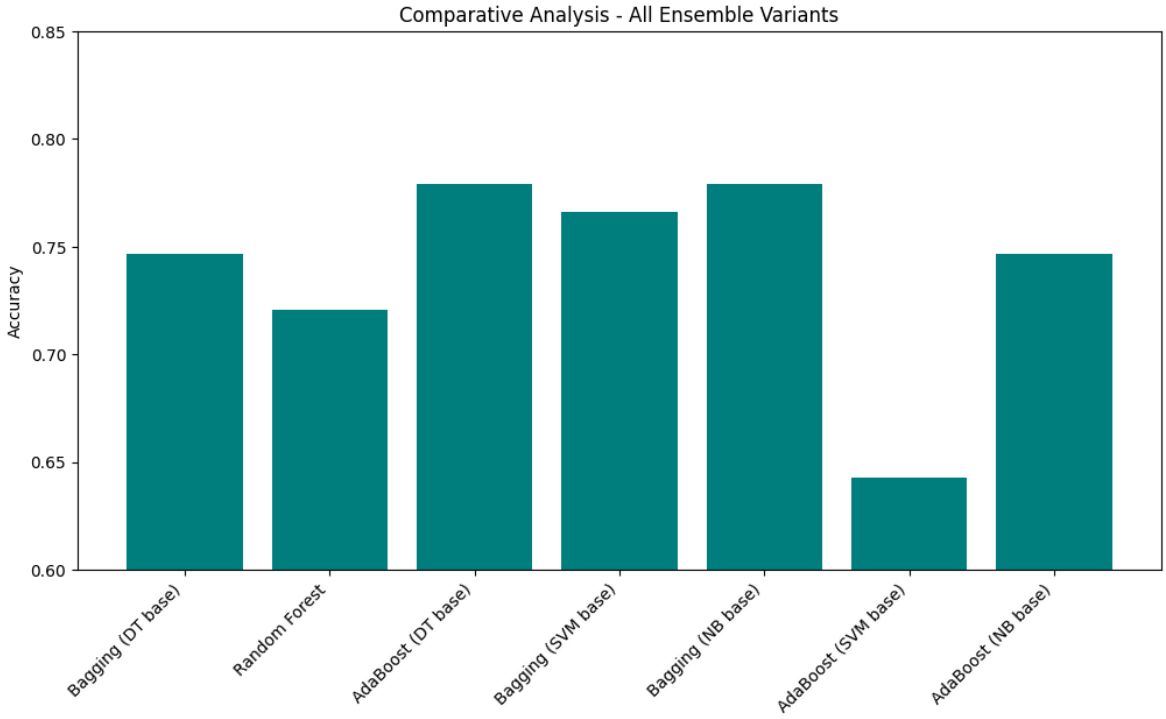
In [9]:
```
# Boosting with SVM as base learner
boost_svm = AdaBoostClassifier(
    estimator=SVC(probability=True),   # AdaBoost needs probability=True for SVM
    n_estimators=50,
    random_state=42,
)
boost_svm.fit(X_train, y_train)
y_pred_boost_svm = boost_svm.predict(X_test)
print("Boosting (SVM base) Accuracy:", accuracy_score(y_test, y_pred_boost_svm))

# Boosting with Naive Bayes as base learner
boost_nb = AdaBoostClassifier(
    estimator=GaussianNB(),
    n_estimators=50,
    random_state=42
)
boost_nb.fit(X_train, y_train)
y_pred_boost_nb = boost_nb.predict(X_test)
print("Boosting (Naive Bayes base) Accuracy:", accuracy_score(y_test, y_pred_boo
```

```
Boosting (SVM base) Accuracy: 0.6428571428571429
Boosting (Naive Bayes base) Accuracy: 0.7467532467532467
```

In [11]:
```
# Final comparison table for all 7 models
comparison = {
    'Model': [
        'Bagging (DT base)', 'Random Forest', 'AdaBoost (DT base)',
        'Bagging (SVM base)', 'Bagging (NB base)',
        'AdaBoost (SVM base)', 'AdaBoost (NB base)'
    ],
    'Accuracy': [
        bag_acc, rf_acc, boo_acc,
        accuracy_score(y_test, y_pred_bagg_svm),
        accuracy_score(y_test, y_pred_bagg_nb),
        accuracy_score(y_test, y_pred_boost_svm),
        accuracy_score(y_test, y_pred_boost_nb)
    ]
}

comp_df = pd.DataFrame(comparison)
```

```
print("\n=== FINAL COMPARATIVE ANALYSIS ===")
print(comp_df.round(4))

# Optional: Bar chart for all 7 models
plt.figure(figsize=(12,6))
plt.bar(comp_df['Model'], comp_df['Accuracy'], color='teal')
plt.xticks(rotation=45, ha='right')
plt.title("Comparative Analysis - All Ensemble Variants")
plt.ylabel("Accuracy")
plt.ylim(0.6, 0.85)
plt.show()
```

```
=== FINAL COMPARATIVE ANALYSIS ===
                Model  Accuracy
0    Bagging (DT base)    0.7468
1        Random Forest    0.7208
2   AdaBoost (DT base)    0.7792
3   Bagging (SVM base)    0.7662
4    Bagging (NB base)    0.7792
5  AdaBoost (SVM base)    0.6429
6   AdaBoost (NB base)    0.7468
```

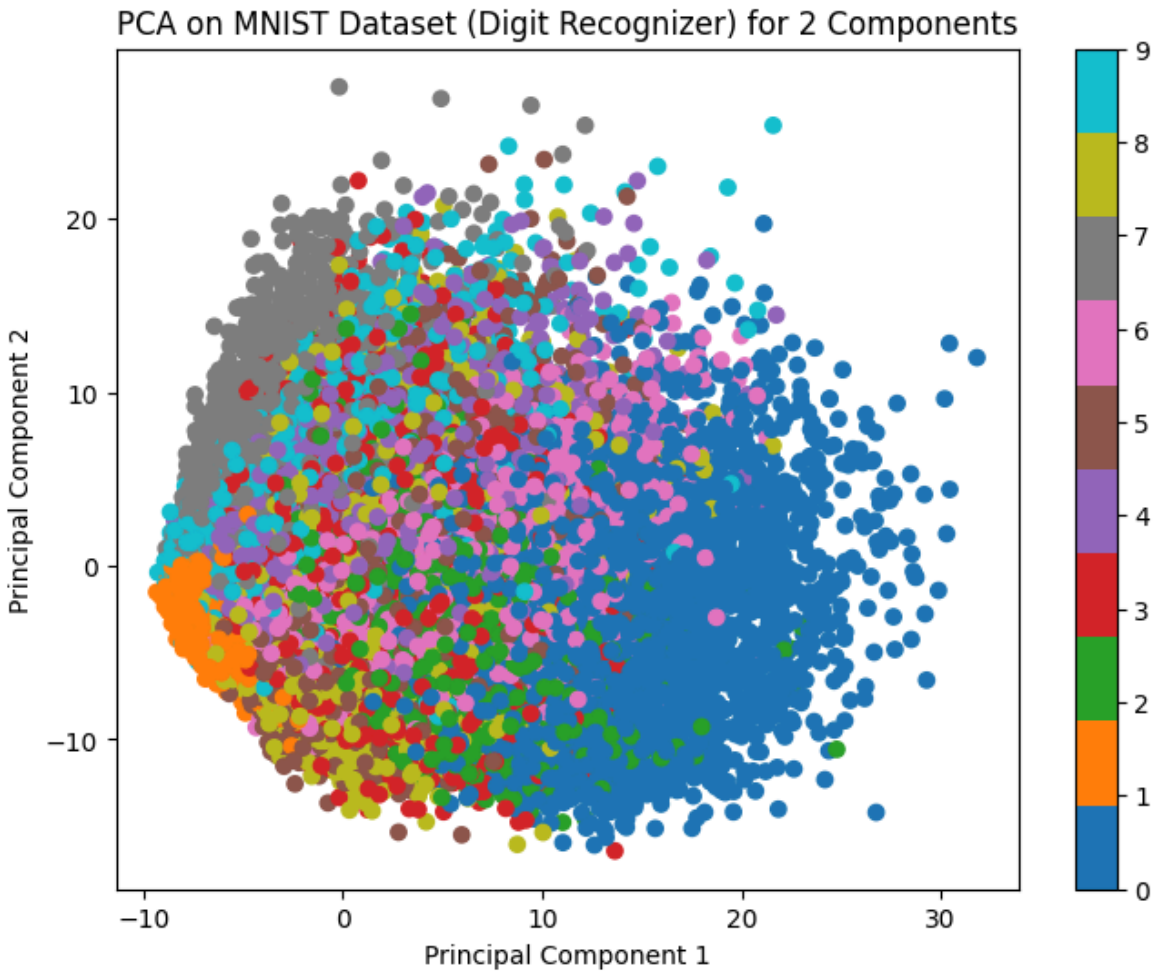


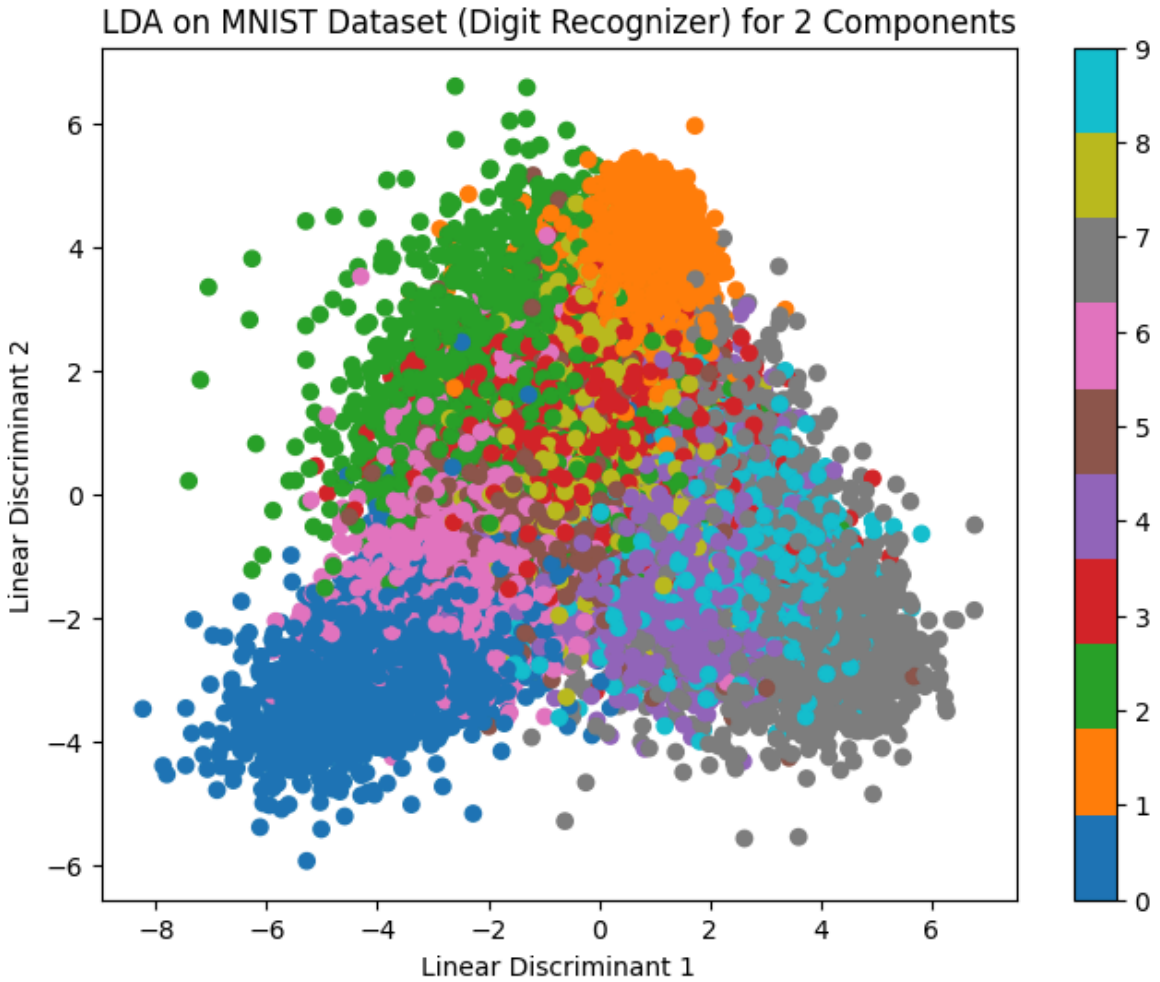
In [ ]:
```
# PCA and LDA Using Digit Recognizer Dataset
```

In [1]:
```
import pandas as pd
df = pd.read_csv('train(DigitRecognizer).csv')
print(df.head())
print(df.shape)

print(df.columns)

#checking for null values
print(df.isnull().sum())

x = df.drop('label', axis=1)
y = df['label']

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

from sklearn.decomposition import PCA
# Here we choose to reduce to 2 dimensions for visualization purposes
pca_2 = PCA(n_components=2)
x_pca_2 = pca_2.fit_transform(x_scaled)
print("Explained variance ratio:", pca_2.explained_variance_ratio_)

import matplotlib.pyplot as plt
plt.figure(figsize=(8, 6))
plt.scatter(x_pca_2[:, 0], x_pca_2[:, 1], c=y, cmap='tab10')
plt.colorbar()
plt.title('PCA on MNIST Dataset (Digit Recognizer) for 2 Components')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
lda_2 = LDA(n_components=2)
x_lda_2 = lda_2.fit_transform(x_scaled, y)

plt.figure(figsize=(8, 6))
plt.scatter(x_lda_2[:, 0], x_lda_2[:, 1], c=y, cmap='tab10')
plt.colorbar()
plt.title('LDA on MNIST Dataset (Digit Recognizer) for 2 Components')
plt.xlabel('Linear Discriminant 1')
plt.ylabel('Linear Discriminant 2')
plt.show()

from sklearn.decomposition import PCA
# Here we choose to reduce to 100 dimensions for visualization purposes
pca_100 = PCA(n_components=100)
x_pca_100 = pca_100.fit_transform(x_scaled)
print("Explained variance ratio:", pca_100.explained_variance_ratio_)

plt.figure(figsize=(8, 6))
plt.scatter(x_pca_100[:, 0], x_pca_100[:, 1], c=y, cmap='tab10')
plt.colorbar()
plt.title('PCA on MNIST Dataset (Digit Recognizer) for 100 Components')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
```

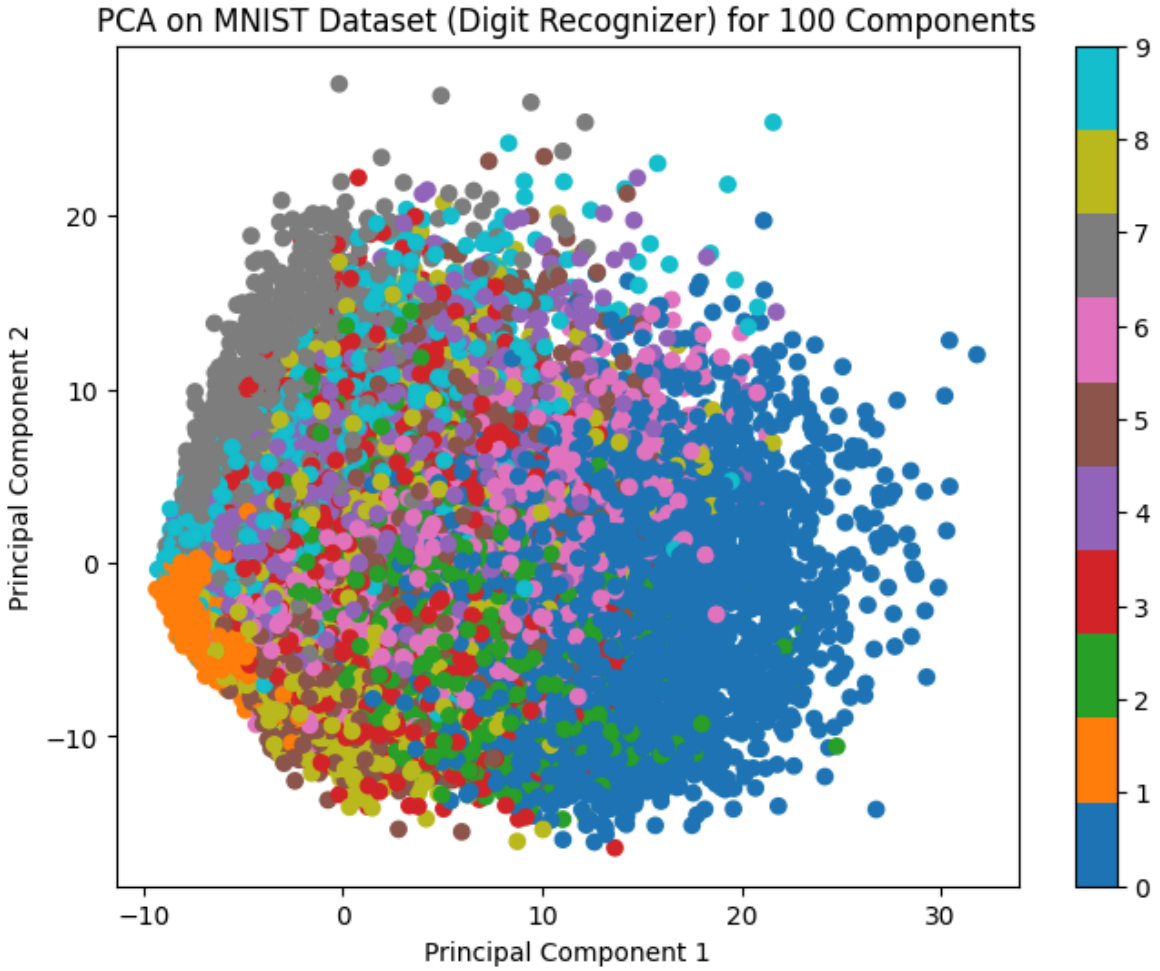
```
plt.show()

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
lda_5 = LDA(n_components=5)
x_lda_5 = lda_5.fit_transform(x_scaled, y)

import matplotlib.pyplot as plt
plt.figure(figsize=(8, 6))
plt.scatter(x_lda_5[:, 0], x_lda_5[:, 1], c=y, cmap='tab10')
plt.colorbar()
plt.title('LDA on MNIST Dataset (Digit Recognizer) for 100 Components')
plt.xlabel('Linear Discriminant 1')
plt.ylabel('Linear Discriminant 2')
plt.show()
```

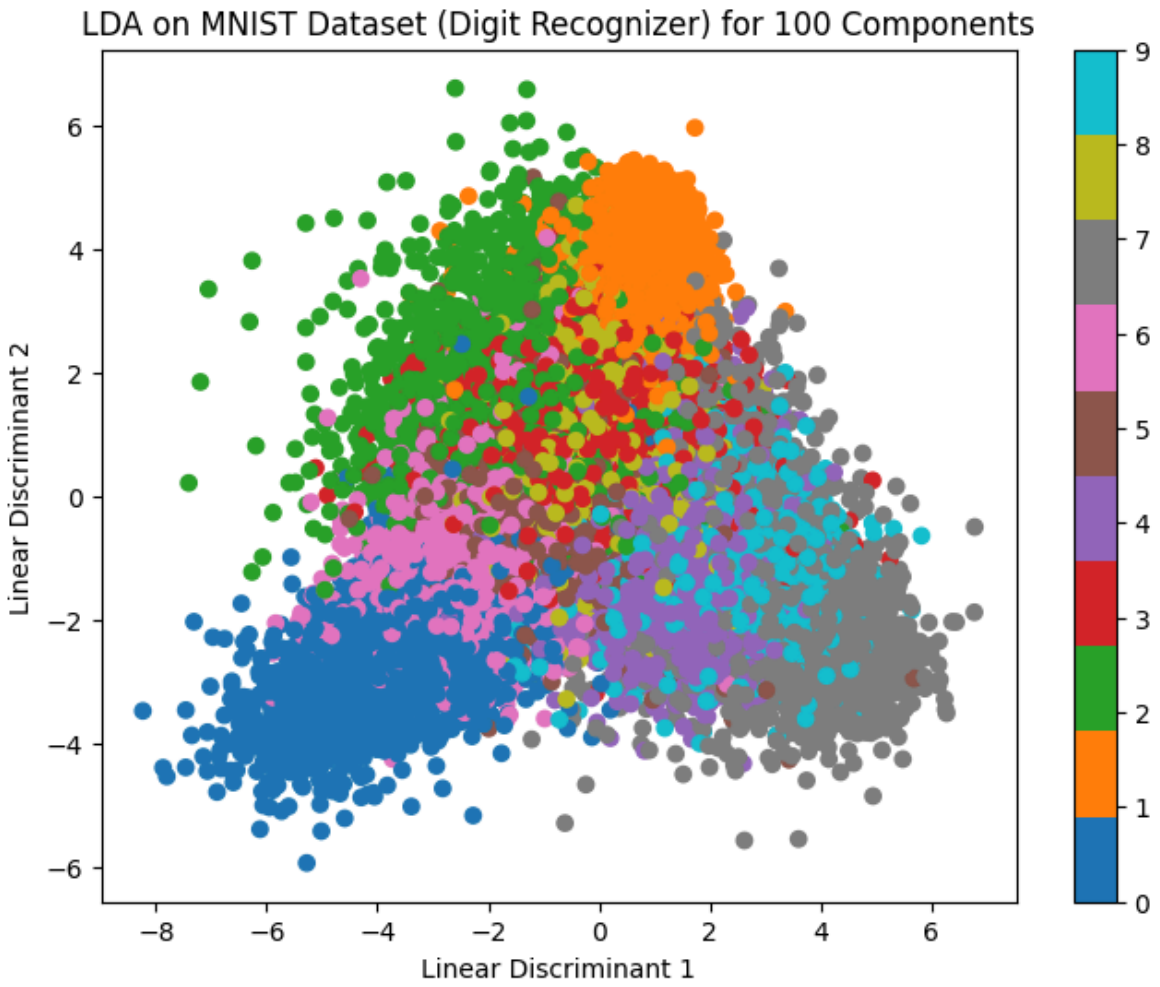
```
   label  pixel0  pixel1  pixel2  pixel3  pixel4  pixel5  pixel6  pixel7  \
0      1       0       0       0       0       0       0       0       0
1      0       0       0       0       0       0       0       0       0
2      1       0       0       0       0       0       0       0       0
3      4       0       0       0       0       0       0       0       0
4      0       0       0       0       0       0       0       0       0

   pixel8  ...  pixel774  pixel775  pixel776  pixel777  pixel778  pixel779  \
0       0  ...         0         0         0         0         0         0
1       0  ...         0         0         0         0         0         0
2       0  ...         0         0         0         0         0         0
3       0  ...         0         0         0         0         0         0
4       0  ...         0         0         0         0         0         0

   pixel780  pixel781  pixel782  pixel783
0         0         0         0         0
1         0         0         0         0
2         0         0         0         0
3         0         0         0         0
4         0         0         0         0

[5 rows x 785 columns]
(42000, 785)
Index(['label', 'pixel0', 'pixel1', 'pixel2', 'pixel3', 'pixel4', 'pixel5',
       'pixel6', 'pixel7', 'pixel8',
       ...
       'pixel774', 'pixel775', 'pixel776', 'pixel777', 'pixel778', 'pixel779',
       'pixel780', 'pixel781', 'pixel782', 'pixel783'],
      dtype='object', length=785)
label       0
pixel0      0
pixel1      0
pixel2      0
pixel3      0
           ..
pixel779    0
pixel780    0
pixel781    0
pixel782    0
pixel783    0
Length: 785, dtype: int64
Explained variance ratio: [0.05747953 0.04111691]
```


PCA on MNIST Dataset (Digit Recognizer) for 2 Components
Principal Component 2
Principal Component 1
20
10
0
−10
−10
0
10
20
30
9
8
7
6
5
4
3
2
1
0


LDA on MNIST Dataset (Digit Recognizer) for 2 Components
Linear Discriminant 2
Linear Discriminant 1
6
4
2
0
−2
−4
−6
−8
−6
−4
−2
0
2
4
6
9
8
7
6
5
4
3
2
1
0

```
Explained variance ratio: [0.05747953 0.04111691 0.03782867 0.02939862 0.02556439
0.02229844
 0.01952552 0.01771605 0.0156266  0.01424956 0.01361009 0.01222541
 0.01135736 0.0111309  0.01050311 0.01012326 0.00951303 0.00934514
 0.00907259 0.00885327 0.00838907 0.00812027 0.00775161 0.00752312
 0.0072769  0.00698756 0.00690455 0.00664924 0.00630675 0.00616301
 0.00610922 0.00597037 0.00577284 0.00573682 0.00564822 0.00546322
 0.0053943  0.00524361 0.00504843 0.0048853  0.00482244 0.00475901
 0.00460008 0.00457629 0.00449584 0.00446722 0.00443837 0.00436837
 0.00432713 0.00427027 0.00419269 0.0041218  0.00402461 0.00399434
 0.00394891 0.00390805 0.00379899 0.00372454 0.00368413 0.00365723
 0.00353278 0.00351088 0.00345414 0.00341394 0.00337784 0.00336477
 0.0033171  0.00329725 0.00320016 0.00316776 0.00312695 0.00311861
 0.00308212 0.00303276 0.00301509 0.0029714  0.00294849 0.00293629
 0.00287848 0.00286906 0.00283171 0.00282885 0.00281701 0.00279215
 0.00276842 0.00276316 0.00273923 0.0027181  0.00268141 0.00265443
 0.00262072 0.00260164 0.00258196 0.00254877 0.00253504 0.00252401
 0.00250074 0.00245803 0.0024436  0.00241387]
```



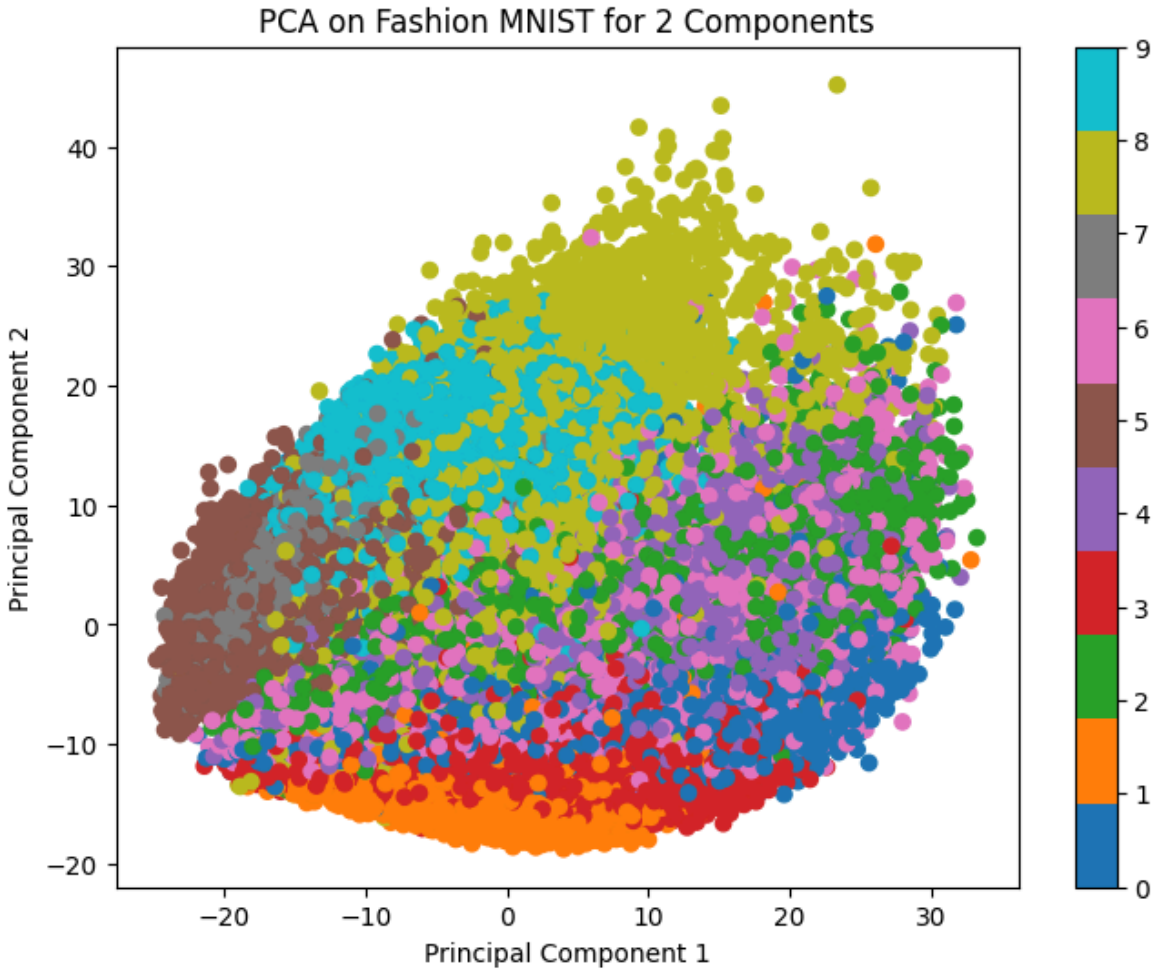


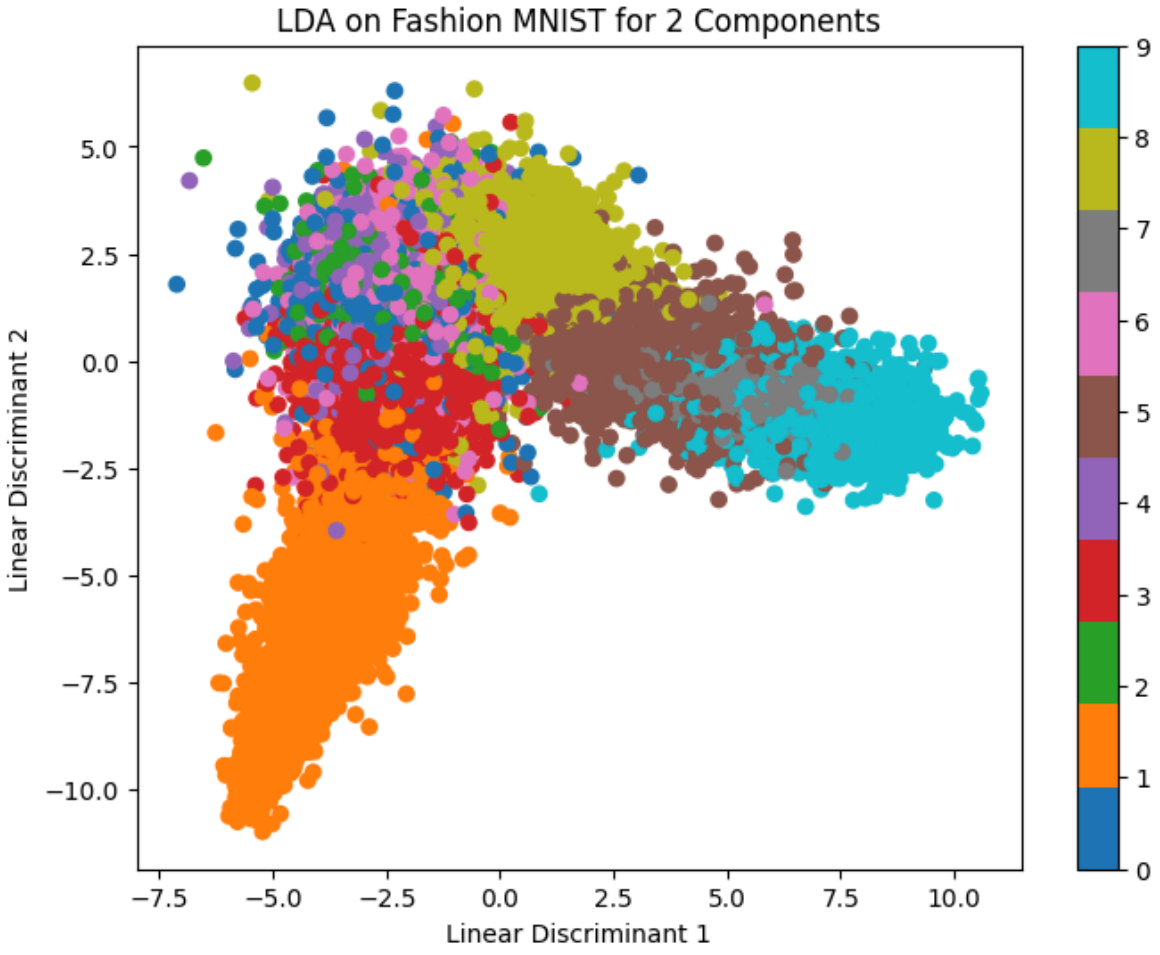
In [ ]:
```
# PCA and LDA with Fashion-MNIST Dataset
```

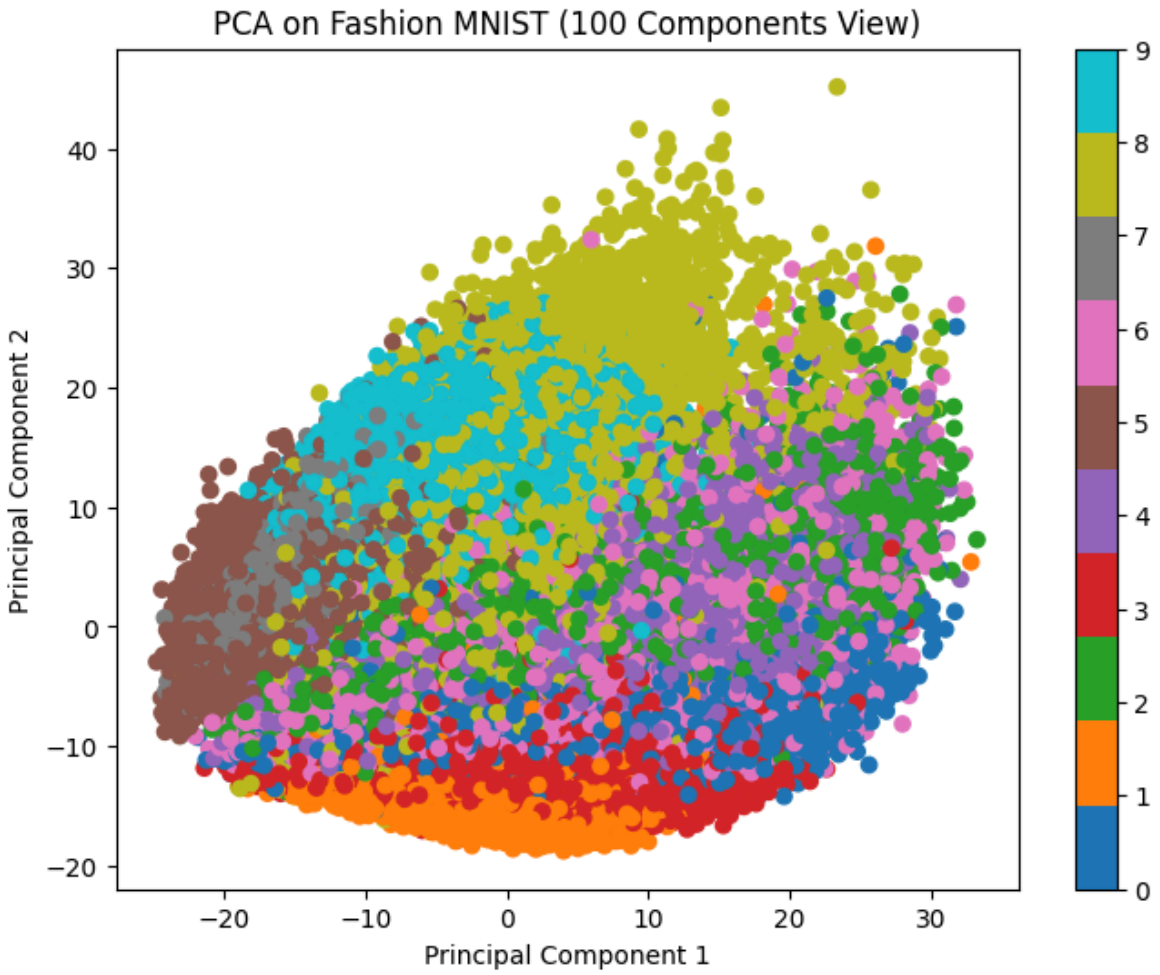
In [24]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

# 1. Load Data
df = pd.read_csv('fashion-mnist_train.csv')
print(df.head())
print(df.shape)

# 2. Check Columns and Nulls
print(df.columns)
print(df.isnull().sum())

# 3. Split Features and Target
x = df.drop('label', axis=1)
y = df['label']

# 4. Standard Scaling
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

# 5. PCA for 2 Components
pca_2 = PCA(n_components=2)
x_pca_2 = pca_2.fit_transform(x_scaled)
```

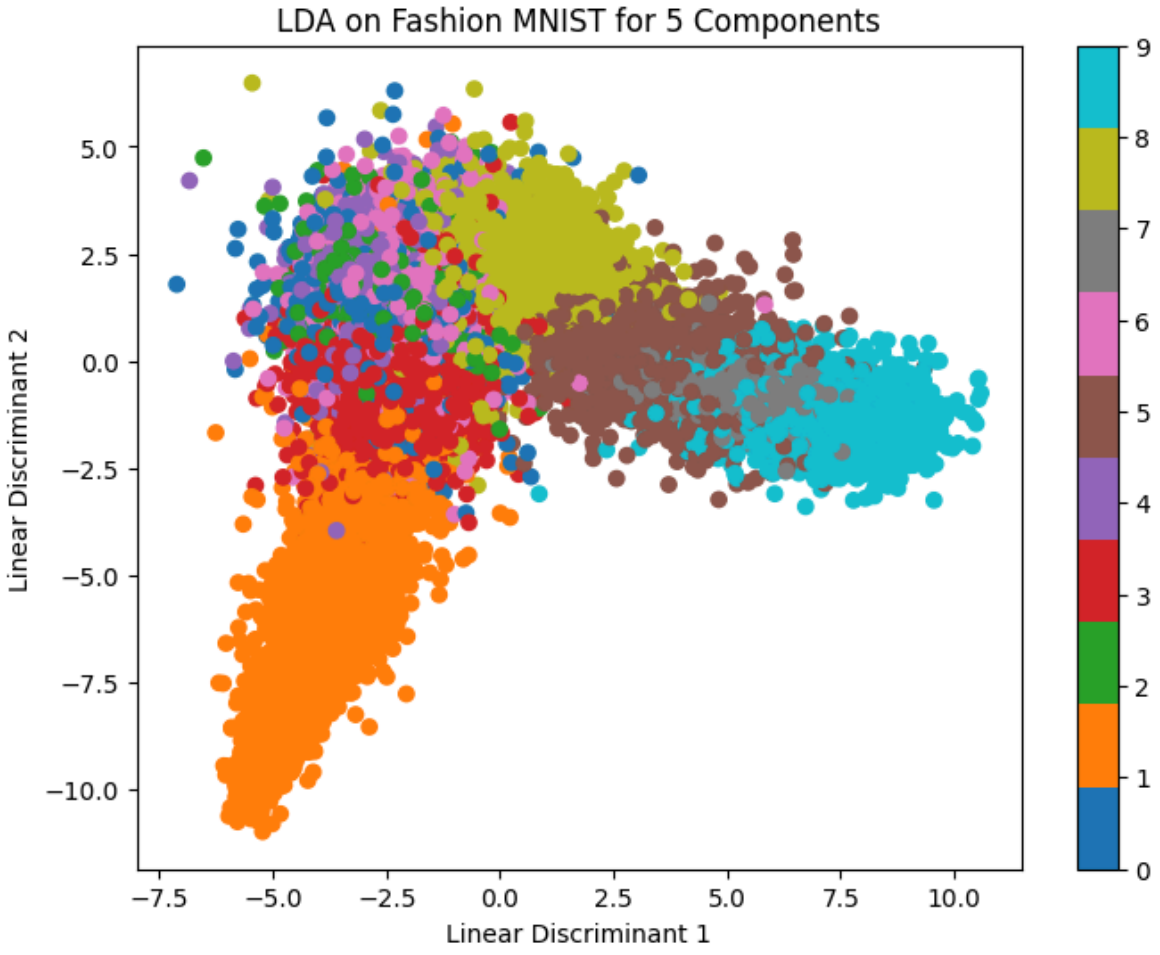
```
plt.figure(figsize=(8, 6))
plt.scatter(x_pca_2[:, 0], x_pca_2[:, 1], c=y, cmap='tab10')
plt.colorbar()
plt.title('PCA on Fashion MNIST for 2 Components')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()

# 6. LDA for 2 Components
lda_2 = LDA(n_components=2)
x_lda_2 = lda_2.fit_transform(x_scaled, y)

plt.figure(figsize=(8, 6))
plt.scatter(x_lda_2[:, 0], x_lda_2[:, 1], c=y, cmap='tab10')
plt.colorbar()
plt.title('LDA on Fashion MNIST for 2 Components')
plt.xlabel('Linear Discriminant 1')
plt.ylabel('Linear Discriminant 2')
plt.show()

# 7. PCA for 100 Components
pca_100 = PCA(n_components=100)
x_pca_100 = pca_100.fit_transform(x_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(x_pca_100[:, 0], x_pca_100[:, 1], c=y, cmap='tab10')
plt.colorbar()
plt.title('PCA on Fashion MNIST (100 Components View)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()

# 8. LDA for 5 Components
lda_5 = LDA(n_components=5)
x_lda_5 = lda_5.fit_transform(x_scaled, y)

plt.figure(figsize=(8, 6))
plt.scatter(x_lda_5[:, 0], x_lda_5[:, 1], c=y, cmap='tab10')
plt.colorbar()
plt.title('LDA on Fashion MNIST for 5 Components')
plt.xlabel('Linear Discriminant 1')
plt.ylabel('Linear Discriminant 2')
plt.show()
```

```
   label  pixel1  pixel2  pixel3  pixel4  pixel5  pixel6  pixel7  pixel8  \
0      2       0       0       0       0       0       0       0       0
1      9       0       0       0       0       0       0       0       0
2      6       0       0       0       0       0       0       0       5
3      0       0       0       0       1       2       0       0       0
4      3       0       0       0       0       0       0       0       0

   pixel9  ...  pixel775  pixel776  pixel777  pixel778  pixel779  pixel780  \
0       0  ...         0         0         0         0         0         0
1       0  ...         0         0         0         0         0         0
2       0  ...         0         0         0        30        43         0
3       0  ...         3         0         0         0         0         1
4       0  ...         0         0         0         0         0         0

   pixel781  pixel782  pixel783  pixel784
0         0         0         0         0
1         0         0         0         0
2         0         0         0         0
3         0         0         0         0
4         0         0         0         0

[5 rows x 785 columns]
(60000, 785)
Index(['label', 'pixel1', 'pixel2', 'pixel3', 'pixel4', 'pixel5', 'pixel6',
       'pixel7', 'pixel8', 'pixel9',
       ...
       'pixel775', 'pixel776', 'pixel777', 'pixel778', 'pixel779', 'pixel780',
       'pixel781', 'pixel782', 'pixel783', 'pixel784'],
      dtype='str', length=785)
label       0
pixel1      0
pixel2      0
pixel3      0
pixel4      0
           ..
pixel780    0
pixel781    0
pixel782    0
pixel783    0
pixel784    0
Length: 785, dtype: int64
```


PCA on Fashion MNIST for 2 Components
Principal Component 2
Principal Component 1
40
30
20
10
0
−10
−20
−20
−10
0
10
20
30
9
8
7
6
5
4
3
2
1
0


LDA on Fashion MNIST for 2 Components
Linear Discriminant 2
Linear Discriminant 1
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
9
8
7
6
5
4
3
2
1
0


PCA on Fashion MNIST (100 Components View)
Principal Component 2
Principal Component 1
40
30
20
10
0
−10
−20
−20
−10
0
10
20
30
9
8
7
6
5
4
3
2
1
0


LDA on Fashion MNIST for 5 Components
Linear Discriminant 2
Linear Discriminant 1
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
9
8
7
6
5
4
3
2
1
0

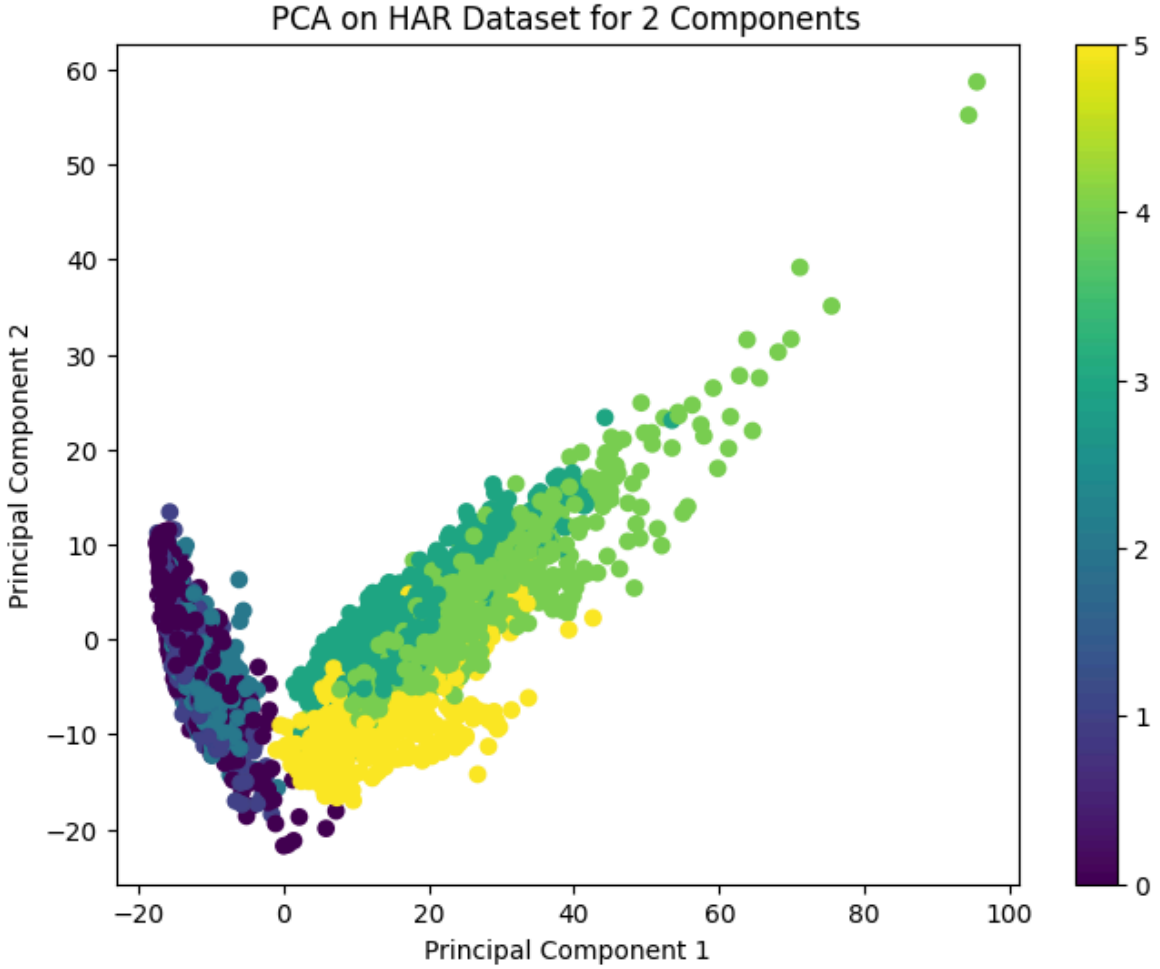
In [22]:
```
# PCA and LDA using Dry Bean Dataset
```

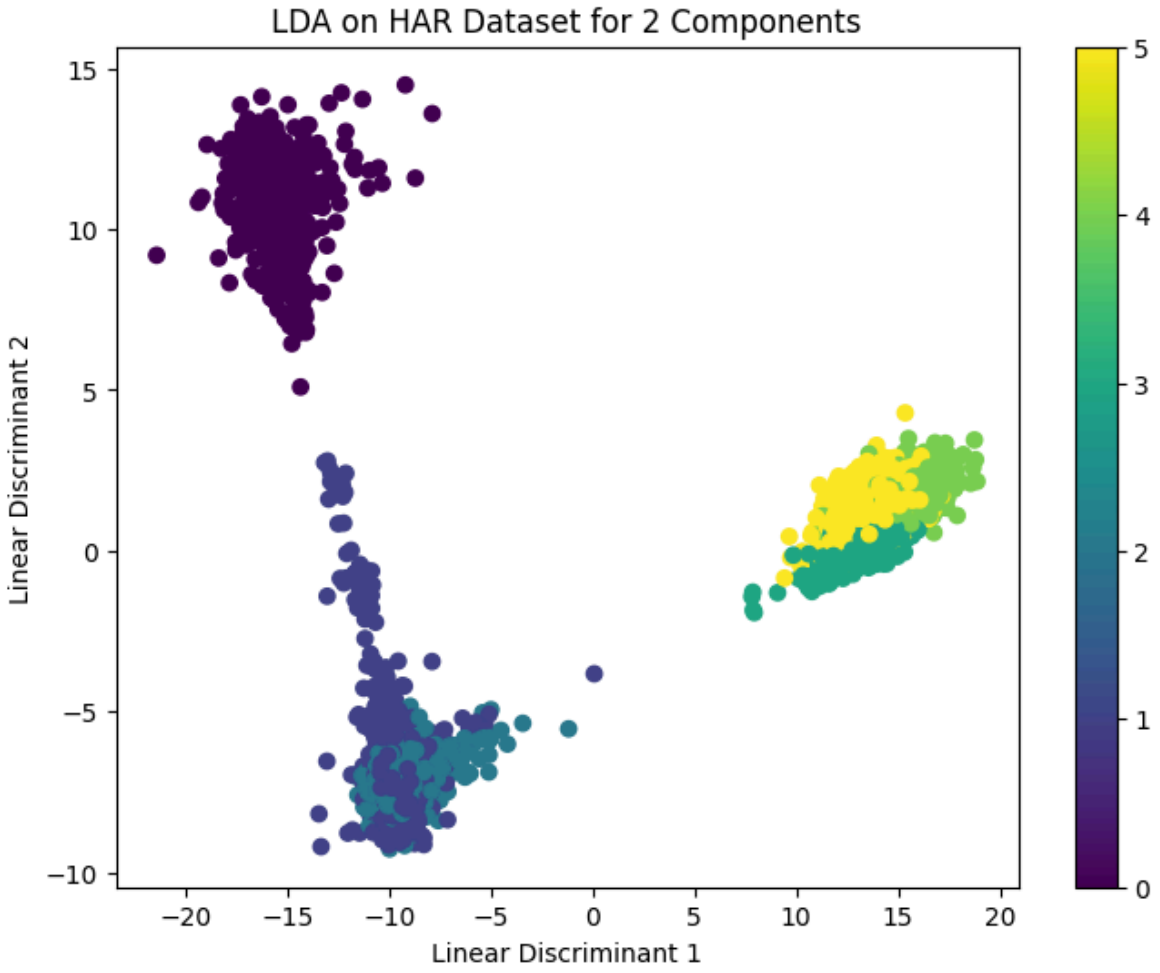
In [26]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

df = pd.read_csv('train(Human_Activity_Recognition).csv')
print(df.head())
print(df.shape)

print(df.columns)
print(df.isnull().sum())

# In this dataset, the target column is usually named 'Activity'
x = df.drop(['Activity', 'subject'], axis=1) # Dropping subject ID as it's not a
y_raw = df['Activity']

# Encode the string labels (Walking, Sitting, etc.) into numbers for plotting
le = LabelEncoder()
y = le.fit_transform(y_raw)

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

# PCA for 2 Components
pca_2 = PCA(n_components=2)
x_pca_2 = pca_2.fit_transform(x_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(x_pca_2[:, 0], x_pca_2[:, 1], c=y, cmap='viridis')
plt.colorbar()
plt.title('PCA on HAR Dataset for 2 Components')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()

# LDA for 2 Components
lda_2 = LDA(n_components=2)
x_lda_2 = lda_2.fit_transform(x_scaled, y)

plt.figure(figsize=(8, 6))
plt.scatter(x_lda_2[:, 0], x_lda_2[:, 1], c=y, cmap='viridis')
plt.colorbar()
plt.title('LDA on HAR Dataset for 2 Components')
plt.xlabel('Linear Discriminant 1')
plt.ylabel('Linear Discriminant 2')
plt.show()

# PCA for 100 Components
pca_100 = PCA(n_components=100)
x_pca_100 = pca_100.fit_transform(x_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(x_pca_100[:, 0], x_pca_100[:, 1], c=y, cmap='viridis')
plt.colorbar()
plt.title('PCA on HAR Dataset (100 Components View)')
plt.xlabel('Principal Component 1')
```

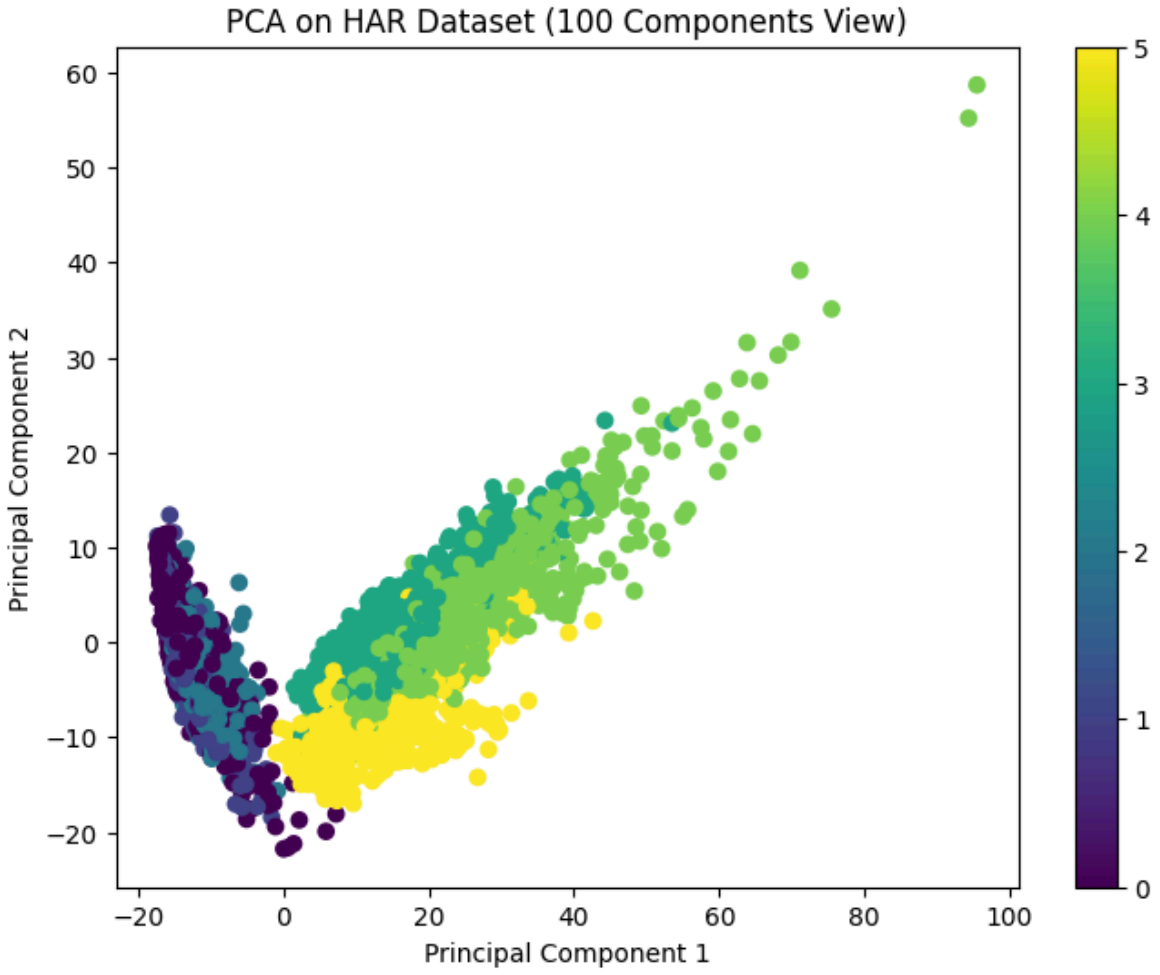
```
plt.ylabel('Principal Component 2')
plt.show()

# LDA for 5 Components
lda_5 = LDA(n_components=5)
x_lda_5 = lda_5.fit_transform(x_scaled, y)

plt.figure(figsize=(8, 6))
plt.scatter(x_lda_5[:, 0], x_lda_5[:, 1], c=y, cmap='viridis')
plt.colorbar()
plt.title('LDA on HAR Dataset for 5 Components')
plt.xlabel('Linear Discriminant 1')
plt.ylabel('Linear Discriminant 2')
plt.show()
```

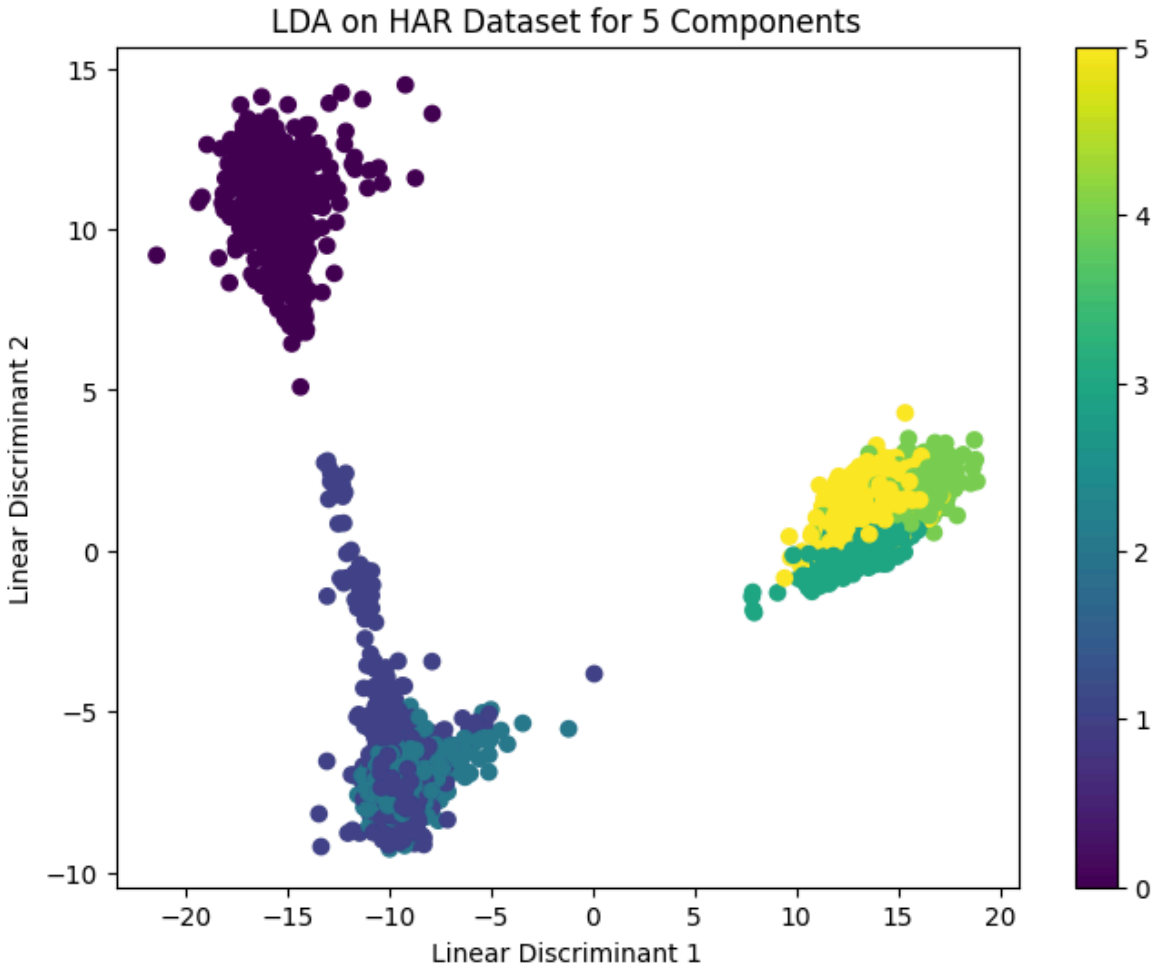
```
   tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z  tBodyAcc-std()-X  \
0           0.288585          -0.020294          -0.132905         -0.995279
1           0.278419          -0.016411          -0.123520         -0.998245
2           0.279653          -0.019467          -0.113462         -0.995380
3           0.279174          -0.026201          -0.123283         -0.996091
4           0.276629          -0.016570          -0.115362         -0.998139

   tBodyAcc-std()-Y  tBodyAcc-std()-Z  tBodyAcc-mad()-X  tBodyAcc-mad()-Y  \
0         -0.983111         -0.913526         -0.995112         -0.983185
1         -0.975300         -0.960322         -0.998807         -0.974914
2         -0.967187         -0.978944         -0.996520         -0.963668
3         -0.983403         -0.990675         -0.997099         -0.982750
4         -0.980817         -0.990482         -0.998321         -0.979672

   tBodyAcc-mad()-Z  tBodyAcc-max()-X  ...  fBodyBodyGyroJerkMag-kurtosis()  \
0         -0.923527         -0.934724  ...                         -0.710304
1         -0.957686         -0.943068  ...                         -0.861499
2         -0.977469         -0.938692  ...                         -0.760104
3         -0.989302         -0.938692  ...                         -0.482845
4         -0.990441         -0.942469  ...                         -0.699205

   angle(tBodyAccMean,gravity)  angle(tBodyAccJerkMean),gravityMean)  \
0                    -0.112754                             0.030400
1                     0.053477                            -0.007435
2                    -0.118559                             0.177899
3                    -0.036788                            -0.012892
4                     0.123320                             0.122542

   angle(tBodyGyroMean,gravityMean)  angle(tBodyGyroJerkMean,gravityMean)  \
0                         -0.464761                            -0.018446
1                         -0.732626                             0.703511
2                          0.100699                             0.808529
3                          0.640011                            -0.485366
4                          0.693578                            -0.615971

   angle(X,gravityMean)  angle(Y,gravityMean)  angle(Z,gravityMean)  subject  \
0             -0.841247              0.179941             -0.058627        1
1             -0.844788              0.180289             -0.054317        1
2             -0.848933              0.180637             -0.049118        1
3             -0.848649              0.181935             -0.047663        1
4             -0.847865              0.185151             -0.043892        1

   Activity
0  STANDING
1  STANDING
2  STANDING
3  STANDING
4  STANDING

[5 rows x 563 columns]
(7352, 563)
Index(['tBodyAcc-mean()-X', 'tBodyAcc-mean()-Y', 'tBodyAcc-mean()-Z',
       'tBodyAcc-std()-X', 'tBodyAcc-std()-Y', 'tBodyAcc-std()-Z',
       'tBodyAcc-mad()-X', 'tBodyAcc-mad()-Y', 'tBodyAcc-mad()-Z',
       'tBodyAcc-max()-X',
       ...
       'fBodyBodyGyroJerkMag-kurtosis()', 'angle(tBodyAccMean,gravity)',
       'angle(tBodyAccJerkMean),gravityMean)',
       'angle(tBodyGyroMean,gravityMean)',
       'angle(tBodyGyroJerkMean,gravityMean)', 'angle(X,gravityMean)',
```

```
       'angle(Y,gravityMean)', 'angle(Z,gravityMean)', 'subject', 'Activity'],
      dtype='str', length=563)
tBodyAcc-mean()-X       0
tBodyAcc-mean()-Y       0
tBodyAcc-mean()-Z       0
tBodyAcc-std()-X        0
tBodyAcc-std()-Y        0
                       ..
angle(X,gravityMean)    0
angle(Y,gravityMean)    0
angle(Z,gravityMean)    0
subject                 0
Activity                0
Length: 563, dtype: int64
```




LDA on HAR Dataset for 2 Components
Linear Discriminant 2
Linear Discriminant 1
15
10
5
0
−5
−10
−20
−15
−10
−5
0
5
10
15
20
0
1
2
3
4
5


PCA on HAR Dataset (100 Components View)
Principal Component 2
Principal Component 1
60
50
40
30
20
10
0
−10
−20
−20
0
20
40
60
80
100
0
1
2
3
4
5


LDA on HAR Dataset for 5 Components
Linear Discriminant 2
Linear Discriminant 1
15
10
5
0
−5
−10
−20
−15
−10
−5
0
5
10
15
20
5
4
3
2
1
0

In [ ]: